

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

DIPLOMSKI RAD br. 1526

**OPTIMIRANJE RASPOREDA RADNIH
NEDJELJA KORIŠTENJEM EVOLUCIJSKIH
ALGORITAMA**

Stjepan Đeleković

Zagreb, lipanj 2026.

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

DIPLOMSKI RAD br. 1526

**OPTIMIRANJE RASPOREDA RADNIH
NEDJELJA KORIŠTENJEM EVOLUCIJSKIH
ALGORITAMA**

Stjepan Đeleković

Zagreb, lipanj 2026.

Hvala mentoru prof. dr. sc. Domagoju Jakoboviću na koordinaciji i pomoći u izradi diplomskog rada te strukturiranom i ležernom pristupu tijekom cijelog mentorstva.

Hvala obitelji na ukazanoj podršci i potpori tijekom cijelog studija bez koje bi studij bio puno zahtjevniji.

Hvala kolegama na suradnji tijekom studija kao i na potpori koja je rezultirala dobrim uspjesima.

Commented [SD1]: UPDATE: zahvala

Sažetak

Optimiranje rasporeda radnih nedjelja korištenjem evolucijskih algoritama

Problem pronalaska dobrog rasporeda radnih nedjelja novi je izazov nastao u Republici Hrvatskoj uvođenjem zakona 2023. godine. Ovaj rad bavi se dizajnom i implementacijom sustava koji bi običnom trgovcu ili trgovačkom lancu omogućio pronalazak optimalnog rasporeda za njihove trgovine, s obzirom na sve druge registrirane trgovine u okolici. Kriteriji koji su modelirani u obzir uzimaju maksimizaciju zarade trgovine kao i zadovoljstvo građana čija je želja svake nedjelje, u svojoj blizini, imati dostupnu trgovinu opće robe. U jezgri sustava nalazi se implementacija orkestriranog genetskog algoritma koji, neovisno o veličini skupa podataka, pronalazi vrlo dobra rješenja u razumnom vremenu koristeći dobro promišljene pozadinske strukture podataka i efikasne algoritme. Sustav je moguće koristiti iz internetskog preglednika što omogućuje korištenje bez naprednog računarskog znanja.

Ključne riječi: genetski algoritam; raspored; trgovine; evolucijsko programiranje; optimizacija

Summary

Optimization of working Sunday schedules using evolutionary algorithms

The problem of finding decent working Sunday schedules is a new problem for Croatian stores introduced with new legislation in 2023. This thesis describes the design and implementation of a system which would help storeowners find optimal schedules for their stores considering other stores in the vicinity. Requirements for an optimal schedule take into consideration the maximization of profits for the store as well as the satisfaction of citizens living in the vicinity of the store whose desire is to have an available store every Sunday. In the center of the system is the implementation of an orchestrated genetic algorithm which finds decent solutions in reasonable time, regardless of the size of the dataset. It does this using a well thought data structures and efficient algorithms. The system can be used from an Internet browser making it possible to use without any advanced computer knowledge.

Keywords: genetic algorithm; schedule; store; evolutionary programming; optimization

Sadržaj

Sažetak.....	1
Summary.....	2
Sadržaj	3
Uvod	6
1. Problem slaganja rasporeda radnih nedjelja	7
1.1. Domena problema.....	7
1.2. Specifikacija problema	8
1.3. Ograničenja.....	8
1.4. Željena svojstva rješenja.....	9
1.5. Prikaz rješenja.....	10
1.6. Slični problemi	11
2. Prikupljanje podataka	12
2.1. Lokacije trgovina.....	12
2.2. „Vrijednost trgovine“	14
2.3. Svojstva podataka	15
2.4. Mane podataka.....	16
3. Genetski algoritam.....	18
3.1. Naivni pristup	18
3.2. Metaheuristike	18
3.3. Darwinova teorija evolucije i raspored radnih nedjelja.....	19
3.4. Genetski algoritam.....	20
3.5. Programski okvir	21
3.6. Procedura općenitog algoritma.....	22
3.6.1. Algoritam grupiranja	25

3.7.	Generator jedinki	26
3.7.1.	Nasumični generator.....	26
3.7.2.	Heuristički generator	26
3.7.3.	Generator na temelju drugih jedinki.....	27
3.8.	Selekcija	27
3.8.1.	Tro-turnirska selekcija.....	27
3.8.2.	Proporcionalna selekcija.....	28
3.9.	Eliminacija.....	28
3.9.1.	Elitistička eliminacija	29
3.9.2.	Geometrijska eliminacija.....	29
3.10.	Križanje	29
3.10.1.	Križanje s k izmjena redaka.....	30
3.10.2.	Križanje s k izmjena stupaca	31
3.11.	Mutacija	31
3.12.	Funkcija dobrote	32
3.12.1.	Funkcija vrijednosti	32
3.12.2.	Funkcija za izračun radijusa	33
3.12.3.	Kombinirana funkcija dobrote.....	34
3.13.	Paralelizacija algoritma	35
4.	Infrastruktura za prezentaciju i automatsku optimizaciju hiperparametara.....	37
4.1.	Korištene tehnologije.....	37
4.2.	Poslužitelj	38
4.3.	Postavljanje aplikacije	39
4.4.	Korisničko sučelje	40
4.4.1.	Popis zadataka	40
4.4.2.	Stvaranje novog skupa podataka (Stores kartica).....	42

4.4.3.	Definiranje hiperparametara algoritma (Settings kartica)	43
4.4.4.	Pokretanje algoritma (Run kartica).....	44
4.4.5.	Pregled rezultat (Results kartica).....	44
5.	Optimizacija hiperparametara i podrezultati	46
5.1.	Skupovi podataka	50
5.2.	Rezultati optimizacije hiperparametara	51
6.	Rezultati.....	53
6.1.	Skupovi podataka	53
6.2.	Rezultati za grad Zagreb.....	54
6.3.	Rezultati za osječko-baranjsku županiju	55
6.4.	Sličnost rješenja.....	56
7.	Zaključak	58
	Literatura	59
	Izjava o korištenju umjetne inteligencije.....	60
	Privitak	61

Uvod

Problem radnih nedjelja novo je nastali problem u Republici Hrvatskoj od proglašenja zakona o izmjenama i dopuni zakona o trgovini 22. ožujka 2023. U članku 2 tog zakona pod točkom (4) piše [2]:

„Trgovac može samostalno 16 nedjelja u godini odrediti kao radne, s time da se trajanju radnog vremena prodajnih objekata iz stavka 1. ovoga članka dodaje 15 sati koje raspoređuje od ponedjeljka do nedjelje.“

Navedeno trgovce dovodi do problema odabira 16 radnih nedjelja za svaku poslovnicu koju posjeduju. Trgovcima je, naravno, želja maksimizirati profit dok je građanima želja da i dalje sve ostane „kako je prije bilo“ i da svake nedjelje imaju neku trgovinu za posjetiti u slučaju potrebe za namirnicama i sl.

Ako funkciju cilja nije moguće modelirati kao linearni problem s obzirom na ulaze, što je u ovome radu slučaj, navedeni problem kombinatorički je NP-potpuni problem, što znači da optimalno rješenje nije moguće pronaći u polinomijalnom vremenu. Trgovci trenutno ne posjeduju nikakve napredne alate za rješavanje ovog problema, već priskaču primitivnim statističkim metodama i intuiciji u nadi da bi pogodili dobar raspored radnih nedjelja.

U ovom radu pokušati ćemo riješiti navedeni problem koristeći metaheuristički pristup u obliku evolucijskog algoritma koji pronalazi dovoljno dobra rješenja u razumnom vremenu bez garancije na optimalnost pronađenih rješenja. Uz implementaciju algoritma, razviti ćemo i interaktivno sučelje namijenjeno uporabi u stvarnom svijetu.

Commented [SD2]: UPDATE: NP potpunost problema ovisi o funkciji cilja

1. Problem slaganja rasporeda radnih nedjelja

Uvođenje zakona o izmjenama i dopuni zakona o trgovini 2023. unijelo je još jednu nijansu kompleksnosti u već do sada zahtjevnu trgovačku industriju u kojoj se sve odluke donose na velikoj skali što znači da svaka odluka utječe ne velik geografski i poslovni prostor što ima značajan utjecaj na financijsko stanje trgovca odnosno trgovačkog lanca. Dosadašnji pristup rješavanju ovog bitnog problema jasno se vidi iz citata šefa udruge malih trgovaca: *"Kako biram 16 radnih nedjelja? K'o kokoš kad nabada zrno. Nekad pogodim, nekad ne."* [1]

Commented [SD3]: UPDATE: Dodana referenca kao sto je trazeno. Osim na ovo mjesto, i na druga mjesta dodane su reference

Ovakav pristup osjeti se i iz perspektive građanina gdje kupac nedjeljom putuje do najbliže benzinske postaje po namirnice koje su im potrebne samo zato što niti jedna od 10 trgovina koje su kupcu bliže od te benzinske postaje ne radi baš te nedjelje. Ovo, uz to što smanjuje zarade trgovaca, stvara i nezadovoljstvo kod građana koji su prije uvođenja zakona navikli da trgovina nedjeljom makar radi skraćeno.

1.1. Domena problema

Zahtjevnost problema proizlazi i iz činjenice da generirani raspored treba „živjeti“ u stvarnome svijetu što čini modeliranje kvalitete rješenja još zahtjevnijim. Naime, zahtjevno je, pa i nemoguće savršeno predvidjeti ponašanje kupaca za danu nedjelju niti uključiti sve faktore u model koji na to utječu. U prikupljenim podacima nikada neće biti modeliran potpuni „profil“ trgovine koji može opisati koliko je ta trgovina popularna među građanima, što ta trgovina točno prodaje, kakva je ona u uspoređi s okolnim trgovinama i sl. Dodatno, u sustav nikada nećemo moći uključiti sve trgovine koje djeluju na nekom području pa time niti diktirati njihovo radno vrijeme koje više nego značajno utječe na kvalitetu našeg rješenja. Ne smijemo zaboraviti spomenuti niti faktore kao što su pozicija trgovine u demografskom smislu, geografskom smislu i točna nedjelja koju razmatramo na koju utječe sezonalnost na koju ponovno utječe geografska lokacija. Demografski trgovina može biti namijenjena školarcima, gdje nije prijeko potrebno da radi nedjeljom (zato što nedjeljom nema škole) ili je to pak trgovina u nekom dijelu grada u koju pretežno zalaze umirovljenici (tada je bitno izabrati dobru organizaciju rasporeda). Geografski trgovina može biti na obali (gdje želimo da trgovine na obali rade tijekom turističke sezone) dok za trgovine na kontinentu želimo da rade tijekom zimskih mjeseci zato što je dio građanstva na odmoru na

obali. Što se sezonalnosti tiče, u te faktore, uz turističku sezonu, možemo uključiti i blagdane i praznike kao i druge kalendarske okolnosti kao što su produženi vikendi i slično.

1.2. Specifikacija problema

Problem odabira radnih nedjelja pripada domeni diskretne kombinatoričke optimizacije. Radi se o pronalasku optimalnog odabira konačnog broja elemenata iz konačnog skupa uz poštovanje niza ograničenja. Kako bi rješavanju problema mogli pristupiti algoritmima, problem je potrebno precizno definirati matematičkim jezikom.

Za početak, definiramo $T = \{t_1, t_2, t_3, \dots\}$ kao skup svih trgovina te skup svih nedjelja $N = \{n_1, n_2, n_3, \dots\}$. Definirajmo i funkciju $v(t_i, n_j, S)$ koju ćemo od sad nazivati funkcijom vrijednosti. U općenitom slučaju, funkcija vrijednosti pridružuje svakoj trgovini t_i vrijednost za danu nedjelju n_j , u slučaju kada u nedjelju n_j , na danom području rade sve trgovine u skupu S , kojem pripada i t_i . Ovu funkciju kasnije ćemo koristiti u svrhu modeliranja funkcija dobre i intuitivno i heuristički pokušati ugraditi željena svojstva rješenja u njeno ponašanje. Funkcija dobre F će kasnije izgledati kao $F(v(t_i, n_j), n_j, S)$ gdje će v biti funkcija vrijednosti, a S skup trgovina koje rade na danom području na nedjelju n_j .

Cilj optimizacije biti će pronaći S^* za svaki n_j tako da suma $F(v(t_i, n_j), n_j, S^*)$ bude maksimalna i da su sva ograničenja zadovoljena.

1.3. Ograničenja

Ograničenja uvode novu dimenziju kompleksnosti u optimizaciju zato što će se tijekom pronalaska boljih rješenja algoritam pronaći i u rješenjima koja nisu validna, npr. trgovina u rješenju radi više od zadanih 16 radnih nedjelja, što otežava pretragu prostora rješenja.

U ovoj optimizaciji susresti ćemo se samo sa tzv. tvrdim ograničenjima, odnosno ograničenjima čije je zadovoljavanje obavezno. U kontekstu optimizacije postoje još i tzv. meka ograničenja čije zadovoljavanje nije obavezno, ali je poželjno. Željena svojstva rješenja mogla bi se tumačiti mekim ograničenjima, no s obzirom da ih ne možemo jednoznačno ispravno definirati, svojstva rješenja u ovome okviru nećemo gledati.

Već spomenuto ograničenje o maksimalnom broju radnih nedjelja glavno je ograničenje kojim se vodi ova optimizacijska procedura i ono će umjesto fiksnog iznosa od 16, kako stoji

Commented [SD4]: UPDATE: Trgovina i nedjelja je više nego tri 😊

u zakonu, biti korišteno kao postavka algoritma s nazivom `WORKING_SUNDAYS`. Ograničenje je zadovoljeno ukoliko svaka od trgovina dana kao ulaz, kroz danu godinu radi točno `WORKING_SUNDAYS` nedjelja.

Osim ograničenja o broju radnih nedjelja, korisniku algoritma omogućiti ćemo postavljanje točnih nedjelja kada dana trgovina mora raditi, odnosno kada dana trgovina ne smije raditi u konačnom rješenju. Ovo omogućuje korisniku našeg algoritma da u rješenje ugradi iskustveno problemsko znanje koje bi inače bilo prezahtjevno ugraditi automatski u algoritma. Primjer za primjenu ovog je da korisnik koji posjeduje trgovine na obali u postavke algoritma stavi da njegove trgovine obavezno rade cijelo ljeto (što bi se odnosilo na barem 10 od ukupno 16 radnih nedjelja za tu trgovinu). Druga primjena je zabrana rada nedjeljama koje su ujedno i blagdani ili praznici iz očitih razloga. Uvođenjem ove vrste ograničenja omogućili smo djelomično samostalno oblikovanje rasporeda što može biti vrijedno potencijalnom korisniku algoritma.

1.4. Željena svojstva rješenja

Kod modeliranja funkcije vrijednosti kao i funkcije dobrote za algoritam vodit ćemo se dvama razumnim kriterijima koja većinski idu jedan uz drugi, ali ih je bitno razlikovati.

1. Kriterij kanibalizacije - Ravnopravna maksimizacija zarade tijekom godine uzimajući u obzir natjecanje među trgovinama
2. Kriterij pokrivenosti - Dobra pokrivenost područja na kojem trgovina djeluje tijekom godine

Svrha prvog kriterija je izaći u susret trgovcu koji koristi naš algoritam dok je svrha drugog kriterija izaći u susret građanima kojima je u interesu da svake nedjelje kroz godinu, u svojoj okolini imaju dostupnu otvorenu trgovinu. Jasno je kako maksimizacija jednog kriterija u isto vrijeme povećava i drugi no u isto vrijeme je jasno kako određene verzije rasporeda idu više pod ruku prvog odnosno drugog kriterija. Npr. trivijalna maksimizacija drugog kriterija podrazumijevala bi barem jednu otvorenu trgovinu na nekom području svake nedjelje dok bi u zaprezi tome bila činjenica da više trgovina treba raditi barem neke nedjelje pa odabir tog rasporeda ovisi o ograničenjima i mijenja rezultate u okviru prvog kriterija.

Kasnije u poglavljima o funkciji vrijednosti i funkciji dobrote pokušati ćemo navedene funkcije modelirati tako da intuitivno maksimiziraju prvi odnosno drugi kriterij, naravno očito je da točne metode za modeliranje navedenoga nema.

Commented [SD5]: Ne razumijem zasto je upitnik iznad funkciji dobrote? Poglavlje kasnije prica o modelu

1.5. Prikaz rješenja

U svrhu jednostavnije definicije prikaza rješenja, definirajmo broj trgovina $|T|$ kao n te broj nedjelja u danoj godini $|N|$ kao m .

Trivijalan prikaz rješenja bi tada bila binarna matrica dimenzije $n \cdot m$, gdje bi retci predstavljali ulazne trgovine dok bi stupci predstavljali sve nedjelje u godini. Ključno ograničenje nad ovom matricom bi tada bilo da svaki redak mora sadržavati točno `WORKING_SUNDAYS` jedinica, dok svi ostali elementi u retku trebaju biti nule. Iz ovakvog modela jednostavno možemo izračunati dimenziju prostora rješenja $D = m \cdot n$, odnosno veličinu prostora rješenja kao $|R| = \binom{m}{\text{WORKING_SUNDAYS}}^n$.

Za jednu trgovinu i 52 nedjelje u godini od kojih 16 mora biti radno (koji slučaj nema smisla optimirati) to iznosi $2.4 \cdot 10^{13}$, dok je za samo 100 trgovina taj broj reda veličine većeg od 10^{1330} iz čega je očito zašto moramo prisegnuti heurističkim odnosno metaheurističkim metodama.

Umjesto trivijalnog prikaza rješenja, u programskoj implementaciji koristiti ćemo drugačiju strukturu za prikaz nekog rješenja, koje će uključivati i ograničenja koja djeluju na taj raspored. Svrha strukture će biti bolja integracija sa genetskim operatorima koji su sposobni automatski čuvati ograničenja bez dodatne provjere i koji svoje funkcije rade vrlo efikasno s obzirom na navedene strukture. S obzirom na to, u programskoj implementaciji jedan raspored prikazan je sa četiri identično indeksirane strukture čija je svrha brzi pristup, brzo uklanjanje i brzo dodavanje elemenata, a te strukture su sljedeće:

- `cluster` – nepromjenjiva lista identifikacijskih oznaka trgovine koja se koristi za preslikavanje indeksa i na konkretnu trgovinu u skupu podataka
- `works` – nepromjenjiva lista koja se sastoji od listi `works[i]` u kojima se nalaze indeksi nedjelja koje i -ta trgovina MORA raditi
- `model` – promjenjiva lista koja se sastoji od listi `model[i]` u kojima se nalaze indeksi nedjelja koje i -ta trgovina RADI specifično u ovom rasporedu

Commented [SD6]: UPDATE: Promijenjeno, sad ima vise smisla

- `antimodel` – promjenjiva lista koje se sastoji od listi `antimodel[i]` u kojima se nalaze indeksi nedjelja koje *i*-ta trgovina MOŽE raditi u ovom rješenju, ali ne radi

Dodatno postoji i struktura koja se u implementaciji naziva `big_matrix`, a koja je istog oblika kao i trivijalni matrični prikaz rješenja samo su umjesto 0 i 1, elementi matrice 0 i unaprijed izračunati $v(t_i, n_j, S)$ s obzirom na stanje u matrici.

Bitno je primijetiti kako se indeksi nedjelja koje su za pojedine trgovine „zabranjene“ kroz ograničenja ne pojavljuju nigdje u ovoj strukturi; **Ideja strukture je da je moguće brzo i efikasno izmjenjivati elemente između model i antimodel pod struktura.**

Commented [SD7]: UPDATE: mijenjati -> izmjenjivati, "exchange elements between model and antimodel structures"

1.6. Slični problemi

Želimo li naš problem usporediti s nekim fundamentalnim problemom, najbolje možemo usporediti sa višestrukim kvadratnim problemom naprtnjače (engl. Quadratic Multiple Knapsack Problem). Ako svaka nedjelja predstavlja jednu naprtnjaču, a trgovine koje rade te nedjelje kao predmete cilj je maksimizirati vrijednost svake naprtnjače. Kvadratna komponenta u našem slučaju zapravo nije kvadratni **interakcijski član vrijednosti predmeta**, već složena funkcija vrijednosti. Višestruku prirodu problema povezujemo s time što optimiramo više nedjelja odjednom. Običan problem naprtnjače rješava se dinamičkim programiranjem u pseudo-polinomijalnom vremenu dok je već kvadratna varijanta problema naprtnjače NP-teška. Ti problemi se u literaturi najčešće rješavaju baš metaheurističkim algoritmima kao što su genetski algoritmi, tabu pretraga i optimizacija rojem čestica.

Commented [SD8]: Predstavlja interakciju između predmeta koji se nalaze u nekon naprtnjaci. Mislim da je točan pojam

2. Prikupljanje podataka

Prije izrade rada, ideja je bila koristiti podatke prikupljene od stvarnih trgovaca kao i podatke javno prikupljene s interneta. Specifikacija prikupljenih podataka jest da moraju sadržavati lokaciju trgovine u formi koordinata kao i neko polje temelju kojeg se može tumačiti važnost, odnosno veličina, trgovine s u odnosu na ostale trgovine u njenoj okolini. Međutim, kontakt s trgovcima je završio neuspješno, vjerojatno iz razloga što trgovci nemaju posebne želje dijeliti svoje poslovne tajne. Stoga, podaci koji su korišteni isključivo su prikupljeni iz javno dostupnih izvora na internetu. Podaci korišteni u ovom radu isključivo su prikupljeni s prostora Republike Hrvatske s obzirom da se cijela domena problema nalazi baš u Hrvatskoj.

2.1. Lokacije trgovina

Kao prvi korak, bilo je potrebno prikupiti lokacije trgovina, točnije koordinate trgovina. U ovu svrhu korištene je projekt otvorenog koda (engl. open source) OpenStreetMap. Cilj OpenStreetMap projekta rekreirati je ekosustav koji nude komercijalni alati kao što su Google Maps ili Apple Maps. OpenStreetMap sadrži besplatne karte kao i podatke o objektima i geografskim značajkama cijelog svijeta. Podaci OpenStreetMap-a su javno prikupljeni (engl. crowd-sourced) pa njihova točnost nije garantirana, no u pravilu su podaci ispravni.

Nad infrastrukturom OpenStreetMap-a postoji i besplatni alat naziva Overpass Turbo. Overpass Turbo omogućuje konstrukciju upita (donekle sličnih SQL) koji se izvršavaju na poslužiteljskoj strani i omogućuju programski dohvat podataka sa OpenStreetMap infrastrukture. U svrhu prikupljanja lokacija trgovina, konstruiran je Overpass Turbo upit koji prikuplja podatke o svim objektima koji sadrže barem jednu od sljedećih ključnih riječi: *market*, *supermarket*, *general*, *grocery*, *convenience*. Rezultat upita je JSON datoteka koja sadrži listu objekata strukture opisane u isječku Kod 1.1.

```
{
  "type": "Feature",
  "properties": {
    "@id": "relation/6308508",
    "addr:city": "Zagreb",
    "addr:housenumber": "22a",
```

```

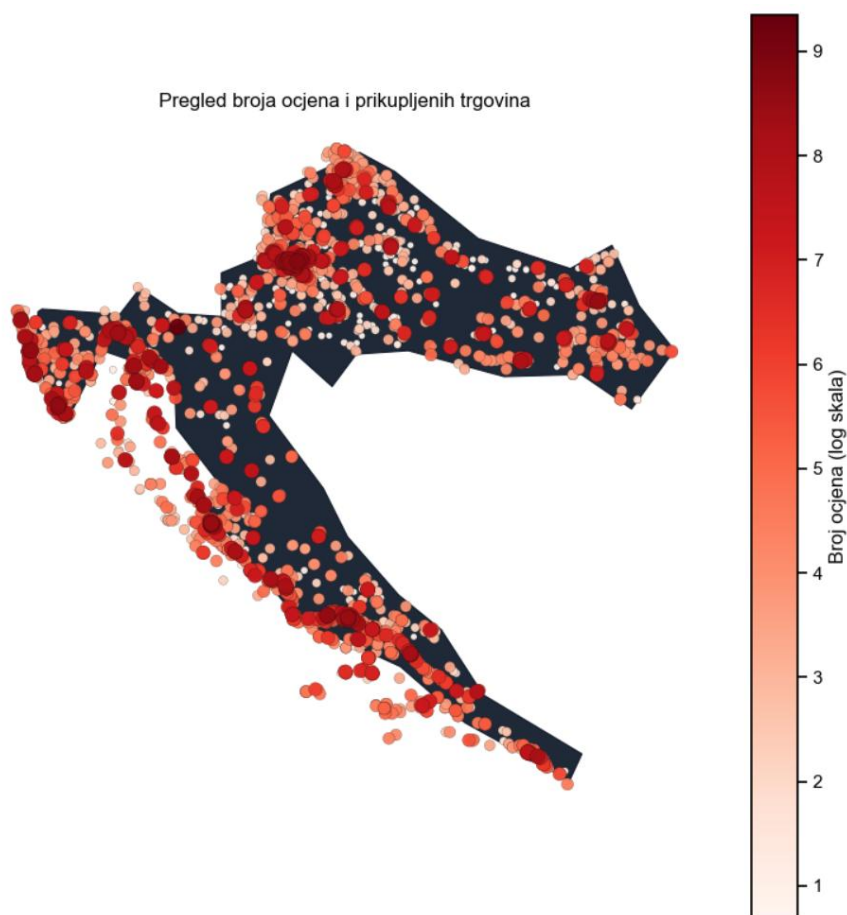
    "addr:postcode": "10000",
    "addr:street": "Ulica Ivana Banjavčića",
    "brand": "Spar",
    "brand:wikidata": "Q610492",
    "brand:wikipedia": "en:SPAR (retailer)",
    "building": "yes",
    "name": "Spar",
    "shop": "supermarket",
    "type": "multipolygon",
    "@geometry": "center"
  },
  "geometry": {
    "type": "Point",
    "coordinates": [
      16.0011734,
      45.8095893
    ]
  },
  "id": "relation/6308508"
}

```

Kod 1 – objekt u kojem se vide sva dostupna polja

Sva navedena polja nisu uvijek prisutna u ovoj strukturi, pa ćemo iz nje koristiti ona polja koja su uvijek tu, a to su identifikator objekta i koordinate objekta. Ostala polja kao što su adresa ime brenda, vrsta trgovine i slično ćemo kasnije koristiti isključivo u informativne svrhe za one trgovine za koje su ta polja specificirana. Sve prikupljene trgovine vidljive su na karti na [slici 1](#) gdje veličina i boja oznake trgovina predstavlja broj ocjena te trgovine.

Commented [SD9]: UPDATE: New map of stores with data



Slika 1 – Trgovine prikupljene na području Republike Hrvatske

2.2. „Vrijednost trgovine“

Glavna ideja definicije vrijednosti trgovine bila je definirati funkciju vrijednosti trgovine kroz parametre koje bi nam dostavili pravi trgovci. To bi tada primarno bili broj izdanih računa, broj prodanih proizvoda i volumen prodaje za danu trgovinu za neku nedjelju (pa čak i bilo koji radni dan). S obzirom da takve podatke nismo uspjeli prikupiti, potrebno je nešto za ovu svrhu pronaći javno dostupno na internetu. Kratkim proučavanjem odlučili smo se koristiti srednjom vrijednosti recenzije i brojem recenzija za specifičnu trgovinu koji su dostupni kroz Google Places aplikacijsko programsko sučelje (engl. API – Application

Programming Interface). Naime, pretpostavka je kako se broj recenzija, pa i srednja vrijednost recenzije, direktno korelira sa očekivanim prometom u toj trgovini. Google Places API omogućuje besplatnu uporabu ako se ne koristi komercijalno i ako se koristi u malim količinama, što je ispalo dovoljno za ovu primjenu. S obzirom da su početne lokacije prikupljene sa OpenStreetMap-a, prvo je bilo potrebno locirati identifikator objekta s obzirom na njegovu geografsku lokaciju (engl. reverse search) pa tek tada dohvatiti dodatne detalje o objektu (srednju recenziju i broj recenzija) s Google API-a. U ove svrhe napravljene su Python skripte koje navedeno rade automatski i rezultate spremaju u JSON formatu. Primjer konačne jedinice podataka može se vidjeti u Kodu 2.

```
"way/29274000": {
    "name": "SPAR supermarket Zagreb Zagrebačka",
    "brand": "Spar",
    "rating": 4,
    "user_ratings_total": 1213,
    "formatted_address": "Zagrebačka cesta 3, 10000, Zagreb, Croatia",
    "coordinates": [
        15.9322674,
        45.8137023
    ]
}
```

Kod 2 – konačni objekt sa podacima jedne trgovine

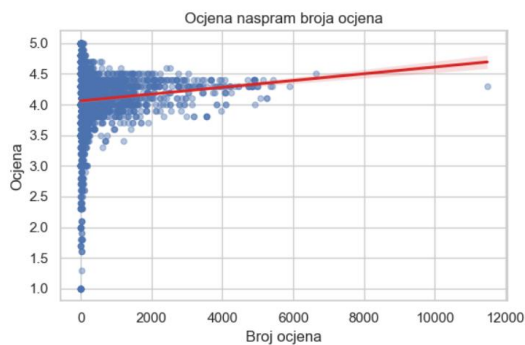
Vidimo da se opisnik jedne trgovine sastoji od njenog identifikatora (u bazi OpenStreetMap), imena trgovine, brenda trgovine ako je isti bio dostupan, srednje recenzije, broja recenzija, formatirane adrese objekta ako je takva postojala te koordinata trgovine koje će se kasnije koristiti u algoritmu i u prezentacijske svrhe.

2.3. Svojstva podataka

Podaci, očekivano, vjerno prikazuju stvarnu distribuciju trgovina u Hrvatskoj, iako zasigurno nisu sve trgovine uključene. U urbanim središtima kao što su Zagreb i ostali veći gradovi nalazi se više trgovina dok su kroz ruralne predjele trgovine rjeđe raspoređene. Za primijetiti je kako će algoritam u ruralnim predjelima (gdje je manja gustoća trgovina) zapravo više optimirati kriterij pokrivenosti dok će u urbanim sredinama, u kojima stanuje

više građana koji trgovinama stvaraju veći promet, algoritam više optimirati kriterij kanibalizacije zato što se mnogo trgovina natječe za kupce na malom području.

Analizom karte trgovina koja prikazuje ovisnost broja recenzija o lokaciji (slika 1), vidimo pristranost u tome da trgovine u većim gradovima generalno imaju veći broj recenzija kao i trgovine na obali. Ovo ima smisla za veće sredine kao i za gradove na obali koje dobivaju recenzije samo tijekom sezone, ali od velike količine stranih građana kojih nema u Hrvatskoj za vrijeme ostalih mjeseci. Ovo će imati utjecaj na funkciju vrijednosti trgovina pa valja imati na umu ovu pristranost. Također, primjenom jednostavne linearne regresije primjećujemo i blagu korelaciju (vidljivo na slici 2) između broja recenzija i srednje recenzije u tome da trgovine sa više recenzija generalno imaju veće recenzije.



Slika 2 – linearna regresija ocjene naspram broja ocjena

Ovi problemi pristranosti samo su dodatno pojačani činjenicom da se podaci prikupljeni s Google API-a ne razlikuju iz nedjelje u nedjelju i time će procjena vrijednosti tijekom sezone za trgovinu obale biti identična procjeni izvan sezone. No ipak, pretpostavka je da ista pristranost vrijedi za sve trgovine na obali i obratno pa bi procedura algoritma na kraju trebala biti ispravna.

2.4. Mane podataka

Osim pristranosti koja je prirodno pristupna u našim prikupljenim podacima postoji i dodatna mana u podacima prikupljenima s interneta. Naime, podaci prikupljeni s Google API-a ne razlikuju se iz nedjelje u nedjelju što uzrokuje pojavu da će funkcija vrijednosti biti nezavisna o nedjelju za koju se promatra. Odnosno za podatke prikupljene s interneta i funkciju vrijednosti vrijedi:

$$v(t, n_i, S) = v(t, n_j, S) \forall i, j$$

iz čega je moguće pojednostavljenje funkcije vrijednosti u kojem vrijedi:

$$v(t, n, S) \equiv v(t, S)$$

Ova pojava će dodatno pojačati greške koje će sezonalna pristranost podataka uvesti. Ipak, pretpostavka je da će pristranost podataka vrijediti za sve trgovine na istom, manjem, području pa ova pristranost konačno ne bi trebala imati velik utjecaj na optimalnost rasporeda.

Dodatna pristranost koja konačno i može imati utjecaja na optimalan raspored je ta što će ljudi rijetko ostaviti recenzije za male trgovine u kojima svakodnevno kupuju kruh, ali će oni i posjetitelji mjesta često ostaviti recenzije za velike šoping centre. Ovo će dodatno i lažno dodati važnost najvećim trgovinama.

Dodatno, valja imati na umu kako su podaci o lokacijama javno prikupljeni i lako je moguće da neke trgovine u skupu podataka više ne postoje kao i da nekih trgovina koje postoje nema.

3. Genetski algoritam

S obzirom na prije izračunatu veličinu prostora rješenja jasno je da ne postoji optimizacijski algoritam koji će optimirati bilo kakvu funkciju dobrote u razumnom vremenu. Naivnim pristupima moguće je generirati dijelove rješenja koji intuitivno idu u korist našim kriterijima pa bi ideje naivnih pristupa valjalo iskoristiti i u našem algoritmu i time suziti pretragu na manji podskup rješenja. Osim toga, koristiti ćemo metaheurističke algoritme, konkretno genetski algoritam koji pronalaze dovoljno dobra rješenja za ovakve probleme, međutim ne garantiraju optimalnost pronađenog rješenja

3.1. Naivni pristup

Kriterij pokrivenosti intuitivno možemo zadovoljiti na način da, uzmemo li grupu trgovina koje djeluju na nekom geografskom području, svakoj nedjelji dodijelimo barem jednu trgovinu koja će te nedjelje raditi. Detalji optimalne pokrivenosti skrivaju se u točnom odabiru trgovina zato što će područje biti bolje pokriveno uzmemo li jedan skup trgovina koje rade te nedjelje nego drugi.

Kako bi pomogli kriteriju kanibalizacije intuitivno se možemo truditi da svaka nedjelja kroz godinu ima podjednak broj trgovina koje rade te nedjelje, ovom raspodjelom izbjegavamo velik broj trgovina koje rade istovremeno što bi u teoriji kanibalizaciju trebalo smanjiti. Optimalna kanibalizacija naravno ovisi o konkretnom odabiru trgovina i načinu na koje interagiraju. Primjerice, možda male trgovine ne žele raditi iste nedjelje kada najveća trgovina u mjestu radi.

Kombinacijom navedenih pristupa dolazimo do metode koja u vrlo brzom vremenu generira raspored sa intuitivnim pravilima. Ideje iz prethodnih pristupa trgovci zasigurno već i koriste kod definicije svojih rasporeda.

3.2. Metaheuristike

Metaheuristike su skup algoritamskih koncepata koje koristimo za definiranje heurističkih metoda primjenjivih na širok skup problema [4]. One koriste iterativne i intuitivne strategije kako bi pretražile prostor rješenja. U metaheuristikama često se koristi slučajnost iz razloga

što su prostori pretraživanja često veliki, kao što je slučaj i u našem problemu, pa nam slučajnost daje mogućnost pronalaska dobrog potprostora rješenja u kojem tada „fokusiranije“ možemo pronaći još bolja rješenja. Baš se metaheuristički algoritmi često primjenjuju na kombinatoričke probleme, gdje klasični algoritmi ne mogu pronaći rješenje u razumnom vremenu zbog NP-težine problema i veličine prostora rješenja. Postoji mnogo inačica metaheuristika te su one često inspirirane prirodnim procesima. Primjeri takvih algoritama, čiji nazivi aludiraju na prirodne pojave, jesu algoritam simuliranog kaljenja, algoritam mravlje kolonije, algoritam roja čestica i evolucijski algoritam koji je inspiriran teorijom evolucije i koji je implementiran u ovome radu.

3.3. Darwinova teorija evolucije i raspored radnih nedjelja

Osnova genetskih algoritama je računalna simulacija prirodnog proces evolucije, čije je temelje postavio biolog Charles Darwin svojom teorijom prirodne selekcije. Naime teorija tvrdi da u prirodi, jedinke koje su najbolje prilagođene svome okolišu imaju najveću šansu preživjeti i prenijeti svoj genetski materijal na sljedeću generaciju. Kroz velik broj generacija, zahvaljujući procesima križanja i mutacija populacija postaje sve naprednija i bolje prilagođena okolišu [6]. Kako bismo ovaj biološki koncept primijenili na naš problem rasporeda potrebno je uspostaviti jasnu analogiju između pojmova u prirodi i koncepata u našem algoritmu.

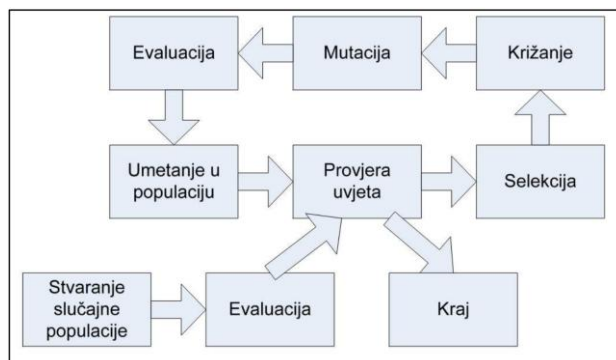
- Okoliš – predstavlja problem koji rješavamo zajedno sa svim njegovim ograničenjima, u našem slučaju pronalazak optimalnog rasporeda radnih nedjelja.
- Kromosom / jedinka – predstavlja jedno potencijalno rješenje problema. U našem slučaju, kromosom je cjelokupni raspored nedjelja za sve trgovine dane kao ulaz u algoritam.
- Gen – najmanja jedinica kromosoma. U našem slučaju, radi li specifična trgovina na specifičnu nedjelju.
- Populacija – skup rješenja koja se natječu za opstanak.
- Prilagodba (funkcija dobrote) – mjera koliko je neka jedinka dobra. Rasporedi koji imaju dobru pokrivenost i nisku kanibalizaciju imati će veći stupanj prilagodbe.

Primjenom evolucijskog pristupa na naš problem ima smisla zbog takozvane hipoteze gradiivnih blokova prema kojoj se optimalno rješenje složenog problema može sastaviti spajanjem manjih, visokokvalitetnih pod-rješenja. Dobra svojstva rješenja preživljavati će

unutar populacije kao genetski materijal zbog operatora križanja dok će operator mutacije nasumično perturbirati rješenje s ciljem proširivanja prostora pretrage.

3.4. Genetski algoritam

Procedura tipičnog genetskog algoritma prikazana je na slici 3. Kod inicijalizacije algoritma potrebno je na neki način stvoriti inicijalnu populaciju. Često je dobra praksa kod generiranja populacije voditi se nekom heuristikom kako bi u populaciji već postojale jedinke koje su dobre u pojedinim dijelovima rješenja. Nakon toga, i svake iteracije, za cijelu populaciju potrebno je izračunati vrijednost funkcije dobrote kako bi operator selekcije znao relativne odnose među jedinkama u populaciji. Operator selekcije potom iz populacije odabire jedinke koje će postati roditelji za iduću generaciju. Odabir roditelja vrši se na stohastički način sa težnjom prema tome da bolje jedinke, prema funkciji dobrote, imaju veću vjerojatnost biti odabrane kao roditelji. Nakon odabira jedinki roditelja, operatorom križanja se izmjenjuju njihovi geni čime nastaju jedinke djeca. Cilj operatora križanja je da iz svakog roditelja iskoristi one gene koji najkvalitetnije doprinose funkciji dobrote (intenzifikacija / eksploatacija). Nastala djeca potom sa nekom vjerojatnosti mutiraju, opet na slučajan način, s ciljem istraživanja prostora rješenja (diverzifikacija / eksploracija). Konačno, jedinke djeca prema nekoj proceduri zamjenjuju druge jedinke u trenutnoj populaciji. Najčešće se u populaciji zamjenjuju baš one najlošije jedinke.



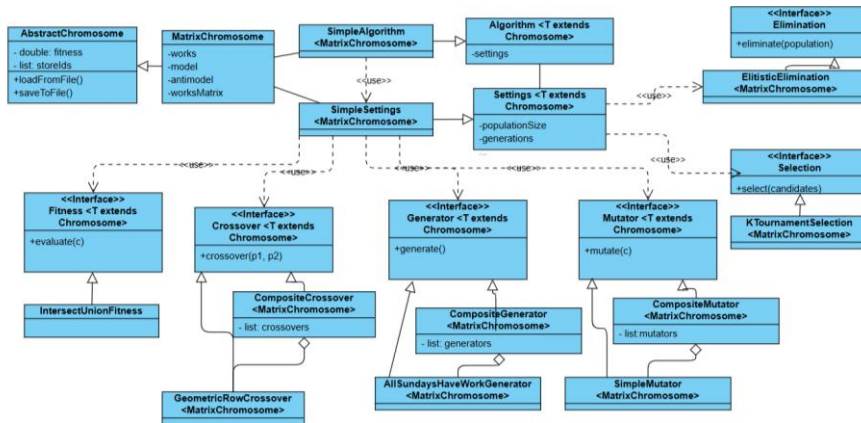
Slika 3 - dijagram jedne iteracije genetskog algoritma [3]

3.5. Programski okvir

Algoritam je u prvotnom obliku implementiran koristeći Python iz razloga što su svi drugi alati vezani uz prikupljanje podataka i pregled rješenja implementirani u Pythonu. Međutim, ispostavilo se da je implementacija u Pythonu prespora za ikakva kvalitetna mjerenja pa je implementacija napisana u Javi. Programski okvir u Javi, koristeći dobre oblikovne obrasce i programerske prakse, omogućuje jednostavnu implementaciju novih genetskih operacija kao i jednostavno uključivanje istih u algoritam. Naime, programsko sučelje implementirano je na način da su dijelovi genetskog algoritma kao što su generator inicijalne populacije, funkcija dobrote i genetski operatori implementirani kao sučelja. Objekt postavki sa bilo kakvim instancama tih sučelja tada se proslijeđuje genetskom algoritmu, implementiranom kao metoda predložka (engl. template method), koji te dijelove koristi bez znanja o njihovom točnom ponašanju.

Kod opće inačice algoritma petlja evolucije izvršava se s tri kriterija zaustavljanja na umu. U postavkama algoritma moguće je postaviti vrijednost za svaki, a radi se o vremenskom kriteriju (engl. wall clock time) koji ograničava vremensko izvršavanje algoritma, postavljenom konačnom broju iteracija te stagnacijskom kriteriju koji završava algoritam ranije ukoliko bolje rješenje od najbolje nije pronađeno određen broj generacija.

Na početku svake iteracije, s obzirom na postavke, odabire se određeni broj jedinki iz populacije koje će preživjeti do iduće generacije. Navedeno se vrši operatorom eliminacije koji ima sučelje kojim na ulazu prima cijelu populaciju, a na izlazu vraća populaciju smanjenu na zadanu veličinu. Operator selekcije u općenitom slučaju ima sučelje da na ulaz primi cijelu populaciju, a na izlazu vraća točno dvije jedinke koje će dalje ići u proceduru križanja. Operator križanja tada ima sučelje koje na ulazu prima točno dvije jedinke roditelja, a na izlazu daje dvije jedinke djece. Razlog stvaranja dvoje djece je što rezultat križanja često ovisi o poretku roditelja pa je ideja stvoriti dvoje djece koja su prema proceduri križanja simetrična. Operator mutacije na ulazi prima jedinku koju je potrebno mutirati te je implementacija mutatora, prikladno, „mutabilna“ što znači da na izlazu ne daje ništa, već izmjenjuje jedinku tzv. „u mjestu“ (engl. in place). Implementacijski detalji kao i postavke svakog od operatora definiraju se u konkretnoj implementaciji sučelja što omogućuje granuliranu definiciju algoritma. Dijagram klasa implementacije vidljiv je na slici 4. Osim jedinki djece, sljedećoj generaciji dodaje se i određeni udio novih, nasumičnih jedinki s



Slika 4 - dijagram klasa implementacije genetskog algoritma

Primijetimo, navedeno je inačica generacijskog genetskog algoritma za koji vrijedi da se stvara „potpuno nova“ populacija djece. Iako u iduću generaciju prenosimo određeni udio najboljih jedinki iz prijašnje generacije, naša implementacija i dalje pripada toj vrsti genetskih algoritma zato što je logika izbacivanja, odnosno zamjene jedinki unutar populacije nezavisna od rezultata genetskih operatora, konkretno operatora selekcije.

Suprotno generacijskom genetskom algoritmu definiran je još i eliminacijski genetski algoritam koji u svakoj iteraciji zamjenjuje samo mali broj jedinki, dok ostatak populacije ostaje netaknut. Za ovakav algoritam najčešće se koristi baš tro-turnirska selekcija gdje se „gubitnik“ turnira zamjenjuje jedinkom djetetom nastalom od „pobjednika“ turnira.

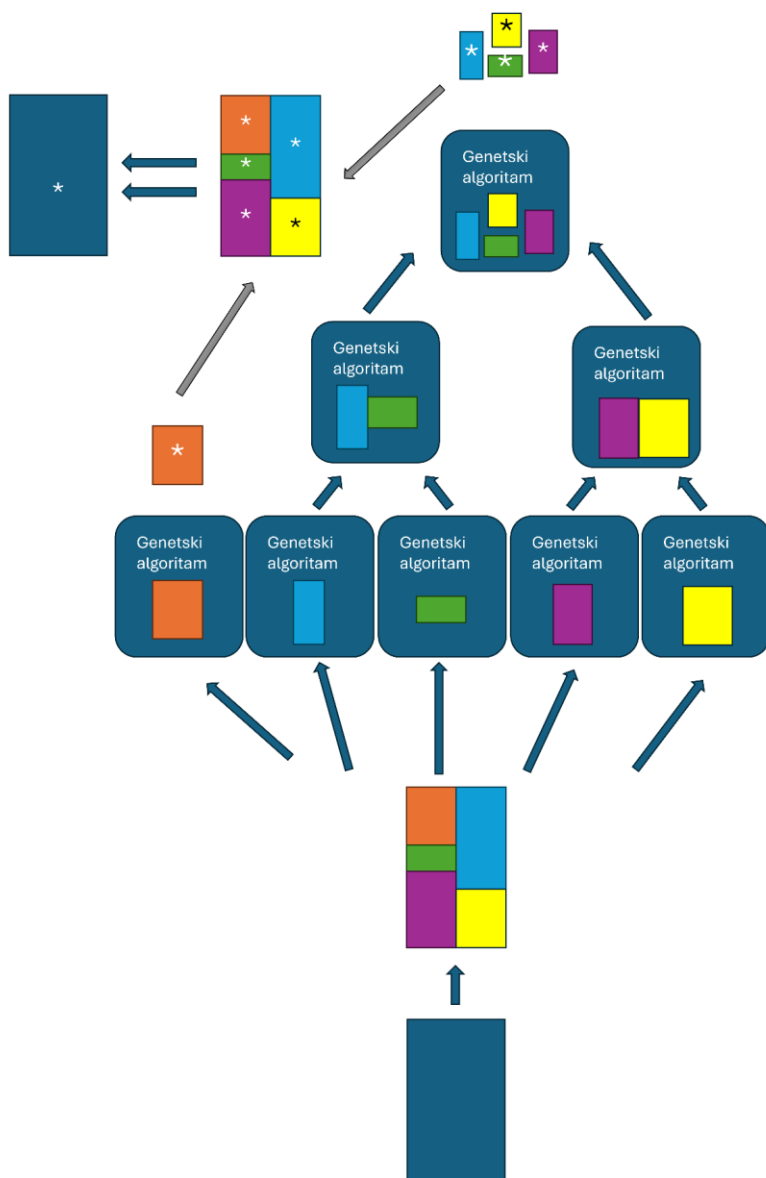
3.6. Procedura općenitog algoritma

S obzirom da je konačan cilj optimirati raspored radnih nedjelja za sve prikupljene trgovine trebamo razmisliti kako dodatno smanjiti pretraživanje. Naime, ako se radi o 4200 trgovina za 2026. godinu, koja ima 52 nedjelje, naše konačno rješenja imati će dimenziju od 218400. No, ipak našem genetskom algoritmu možemo pomoći ako primijetimo kako neke trgovine jednostavno nema smisla niti potrebe optimirati istovremeno zato što intuitivno radno

vrijeme trgovina u Zagrebu nema nikakve veze s radnim vremenom trgovina u Splitu, pa čak niti s radnim vremenom trgovina u obližnjoj Velikoj Gorici.

Dodatno, s obzirom da su rasporedi kao takvi između trgovina nezavisni, složeni problem koji se sastoji od velike količine trgovina možemo rješavati u nekoliko koraka na način problem od recimo 100 trgovina, rastavimo na 10 problema sa 10 trgovina koji postepeno spajamo u 5 problema sa 20 trgovina koristeći već optimalnu početnu populaciju i tako dalje do konačnog problema od 100 trgovina. Jasniji princip grananja prikazan je na slici 5. Kod metaheurističkih algoritama za očekivati je kako će algoritam za manji problem pronaći rješenje bliže optimumu nego za veći problem. Isto je za očekivati kako će trgovine koje su udaljenije od neke trgovine manje utjecati na njen raspored radnih nedjelja.

U tu svrhu, definirati ćemo neki algoritam grupiranja (engl. clustering) koji će naše trgovine grupirati u grupe na način da to ima smisla. Algoritam treba imati postavke kojima se postavlja najveća dozvoljena udaljenost između trgovina kada ima smisla grupirati ih te dodatno postavku koja određuje najveću veličinu grupe. Kod procedure spajanja više grupa uzeti ćemo u obzir ima li ih smisla spojiti i optimirati ponovno kroz konfiguracijsku postavku.



Slika 5 - prikaz procedure orkestriranog genetskog algoritma (početak odozdo)

3.6.1. Algoritam grupiranja

Implementirana je jednostavna pohlepna heuristička procedura za grupiranje sa sljedećim pseudo kodom Kod 3.

```
Funkcija GenerirajGrupe(trgovine, maxClusterSize, maxDistance):  
  
    grupe = nova prazna lista  
    neraspoređeno = kopija liste trgovine  
  
    Dok neraspoređeno NIJE prazno Čini:  
        prva = NasumičnoOdaberiIUkloni(neraspoređeno)  
        trenutnaGrupa = { prva }  
        Dok Veličina(trenutnaGrupa) < maxClusterSize I neraspoređeno NIJE prazno Čini:  
            kandidat = PronađiNajbližuTrgovinu(prva, neraspoređeno)  
            udaljenost = IzračunajUdaljenost(prva, kandidat)  
  
            Ako je udaljenost > maxDistance Tada:  
                Prekini unutarnju petlju  
  
            Dodaj kandidat u trenutnaGrupa  
            Ukloni kandidat iz neraspoređeno  
  
        Dodaj trenutnaGrupa u grupe  
  
    Vrati grupe
```

Kod 3 – pseudokod algoritma grupiranja

Procedura pohlepno puni grupe (engl. cluster) s najbližim trgovinama sve dok je najbliža trgovina unutar najveće dozvoljene udaljenosti. Rezultat ove procedure nije savršeno grupiranje u tradicionalnom smislu, no za trenutne zahtjeve ona je dovoljno dobra. Naime, valja primijetiti kako nije potrebno savršeno grupiranje zato što ćemo kasnije svakako spajati više grupa i optimirati ih zajedno. Kod ovog pristupa može se desiti da se u isto vrijeme dio rješenja (rasporeda) pokvari dok se drugi, odvojeni dio, rješenja poboljša. No, navedeno je općenita mana genetskih algoritama zato što oni optimiraju rješenje u cjelini, a ne dio po dio. Ipak, ideja genetskih algoritama je da se dobra svojstva dijelova rješenja prenose iz

Commented [SĐ10]: PITANJE: Koje su negativne strane ovog pristupa? Algoritam može dati ukupno bolje rješenje zbog poboljšanog rasporeda jedne velike grupe, ali se istovremeno može pogorsati raspored neke manje grupe.

ODGOVOR: Cilj optimizacije maksimizirati je funkciju cilja za problem u onoj konačnoj, najvećoj grupi. Ne zanima nas ako se u procesu poboljšanja velike, konačne grupe kvari raspored koji bi funkcija cilja dala u nekom prethodnom koraku. Nuspojava je da se neka udaljenija grupa slučajnom mutacijom pokvari u istom trenutku dok se velika grupa popravi no ovo je istina za SVAKI genetski algoritam. GA ne zanimaju optimalni dijelovi rješenja, već optimalno cijelo rješenje. S obzirom da je mana općenito povezana s GA mislim da detaljnije od ovog ne treba analizirati.

generacije u generaciju što bi na kraju rezultiralo optimalnim rasporedom obje skupine trgovina

3.7. Generator jedinki

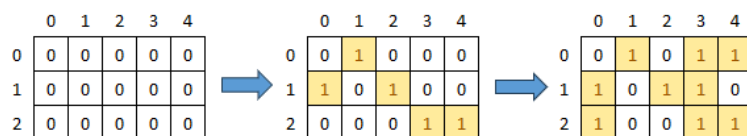
Generator je komponenta algoritma koja sudjeluje u generiranju početne populacije kao i u punjenu populacije sa novim jedinkama. Njeno sučelje sastoji se od metoda `generate` i `generateMany` koje vraćaju jedinku odnosno listu jedinki pozivatelju. Što se primjene generatora tiče, u implementaciji razlikujemo općeniti generator i generator koji stvara nove jedinke na temelju drugih jedinki. Generator koji koristi druge jedinke koristi se nakon spajanja grupi trgovina u veće grupe kako bi se što više iskoristila dobra svojstva prije dobivenih rješenja. Implementirana je također i kompozitna varijanta generatora koja svaki od generatora djece koristi s nekom vjerojatnosti za generiranje nove jedinke.

3.7.1. Nasumični generator

Nasumični generator, kao što se može zaključiti iz naziva, nasumično stvara novu jedinku na način da premješta nedjelje iz antimodela u model sve dok broj nedjelja koje trgovina radi te godine ne bude jednak `WORKING_SUNDAYS`. Potpuno stohastičan generator veoma je dobar u pokrivanju prostora pretraživanja, međutim možda rezultira u jedinkama koje su mnogo lošije od onih nastalih korištenjem drugih generatora. Ovaj generator je možda dobro koristiti zajedno s heurističkim generatorom gdje se potpuno nasumični generator poziva s malom vjerojatnosti.

3.7.2. Heuristički generator

Heuristički generator inspiriran je pohlepnim pristupom. Kod generiranja nove jedinke, prvo se svakoj nedjelji dodjeljuje barem jedna nasumična trgovina iz grupe koja radi te nedjelje kao što je prikazano na slici 6. Na ovaj način intuitivno i trivijalno zadovoljavamo kriterij pokrivenosti. Ostatak rješenja generira se potpuno nasumično. Jedinke generirane heurističkim generatorom sigurno će biti bolje u općenitom slučaju od onih generiranih potpuno nasumičnim generatorom.



Slika 6 - procedura heurističkog generatora

3.7.3. Generator na temelju drugih jedinki

Kao što se može zaključiti iz naziva, ovaj generator stvara nove jedinke na temelju drugih jedinki koje sadrže potrebne trgovine zajedno s njihovim rasporedima. On je implementiran u dvije varijante, determinističkoj i stohastičkoj. U stohastičkoj varijanti, raspored za neke trgovine se ipak generira potpuno nasumično kako bi se održala nedeterministička priroda generatora. Ovaj generator koristi se kada rješavamo složeniji problem prije rješavanja kojeg smo riješili pripadne potprobleme pa ovaj generator koristimo za iskorištavanje rješenja potproblema. Ovaj generator u sučelju ima definiranu postavku p , koja predstavlja vjerojatnost slučajnog permutiranja za neku trgovinu.

3.8. Selekcija

Selekcija je također implementirana generički. Sučelje operatora je takvo da na ulazu prima cijelu populaciju, a kao izlaz daje točno dvije jedinke koje bi se trebale koristiti kao roditelji kod operatora križanja. Generalno pravilo operatora selekcije je da jedinke koje su bolje prema funkciji dobrote imaju veću vjerojatnost biti odabrane kao roditelj. Dobro svojstvo selekcije je također da sve jedinke imaju barem neku vjerojatnost biti odabrane. Pojam koji vezemo uz operator selekcije jest selekcijski pritisak. Selekcijski pritisak predstavlja koliko je za neku jedinku zahtjevno da postane roditelj, odnosno opisuje koliko je jako distribucija odabira roditelja otežana u korist najboljih jedinki.

3.8.1. Tro-turnirska selekcija

Najčešće korišteni oblik selekcije je turnirska selekcija. Turnirska selekcija sa veličinom turnira k odvija se na način da točno dva najbolja od nasumično odabranih k jedinki iz populacije budu odabrani kao roditelji. Najčešće korištena veličina turnira je 3 pa se ta inačica turnirske selekcije poznato naziva i tro-turnirska selekcija. Turnirska selekcija najčešće je korišteni oblik selekcije zbog jasne intuicije i jednostavne implementacije, a u

konačnici daje vrlo dobre rezultate. Parametar `k`, odnosno `tournamentSize` je postavka ovog operatora selekcije koju ćemo kasnije mijenjati.

3.8.2. Proporcionalna selekcija

Još poznata kao i selekcija ruletom (engl. roulette-wheel selection) operator je selekcije kod koje je vjerojatnost odabira svake jedinke da postane roditelj proporcionalna s kvalitetom jedinke. Česta implementacija je da je težina vjerojatnosti odabira jedinke jednaka točno dobroti te jedinke, međutim u našem problemu ispada kako su iznosi naše funkcije dobrote podjednaki između jedinki pa navedeno ne daje dobre rezultate. Umjesto takve proporcionalnosti, implementirana je proporcionalna selekcija na temelju ranga jedinke u odnosu na druge jedinke u populaciji. Drugim riječima, vjerojatnost odabira i -te jedinke jednaka je

$$p_i = \frac{\frac{1}{i}}{\sum_{k=i}^n \frac{1}{k}}$$

Primijetimo kako za ovaj operator nemamo način za diktiranje selekcijskog pritiska. Ako bi to htjeli, selekcijski pritisak α na ovaj bi postupak mogli primijeniti na sljedeći način

$$p_i = \frac{\frac{1}{i^\alpha}}{\sum_{k=i}^n \frac{1}{k^\alpha}}$$

međutim, iz prijašnjeg iskustva promjena selekcijskog pritiska ne utječe značajno na rezultate genetskog algoritma.

3.9. Eliminacija

Iz razloga što je genetski algoritam implementiran na što je moguće više generički način, u našu implementaciju treba uvesti i operator eliminacije koji će određivati koje jedinke neće preživjeti u iduću populaciju. Bez operatora eliminacije način na koji se jedinke uklanjaju iz populacije bio bi fiksni. Operatorom eliminacije možemo simulirati proceduru kojom se jedinke uklanjaju koristeći tro-turnirsku selekciju kao i proceduru kojom se jedinke uklanjaju u jednostavnom generacijskom genetskom algoritmu. Uz operatore selekcije i eliminacije vezemo pojam elitizma. Elitizam je broj najboljih jedinki koje će preživjeti u iduću generaciju. U praksi se pokazalo da je elitizam od barem jedne jedinke potreban zato što

Commented [SB11]: Predložili ste windowing gdje se fitness skalira na linearni interval [1, s]. Previdio sam to. Ali mislim da je i ova selekcija proporcionalna rangu isto dobra. Previše truda bilo bi sada ponovno dodavati i raditi HPO optimizaciju s dodanim Windowed proportional selectorom.

gubljenje trenutnog najboljeg rješenja iz populacije ne daje dobre rezultate, odnosno ponekad dovodi do divergencije algoritma. Sučelje operatora je takvo da na ulazu prima cijelu populaciju, a na izlazu daje populaciju bez jedinki koje su eliminirane.

3.9.1. Elitistička eliminacija

Implementirana je jednostavna elitistička eliminacija, odnosno procedura slična jednostavnoj inačici generacijskog genetskog algoritma. Elitistička eliminacija jednostavno kopira najboljih `elitism` jedinki u iduću populaciju. Navedeni elitizam je u konačnici i postavka ovog eliminacijskog operatora.

3.9.2. Geometrijska eliminacija

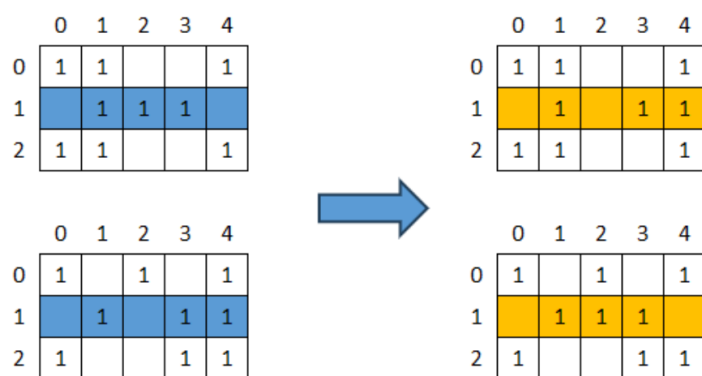
Osim elitističke eliminacije implementirana je i geometrijska eliminacija koja bi trebala oponašati rezultate koje bi dalo tro-turnirsko uklanjanje jedinki iz populacije. Kod geometrijske eliminacije, prvo se najbolja jedinka kopira u iduću generaciju, nakon toga se udio jedinki definiran parametrom `survivalRate` iz trenutne populacije kopira u iduću na način da svaka iduća jedinka ima geometrijski manju vjerojatnost biti kopirana u iduću generaciju. Ovaj operator eliminacije ima svojstvo da ponekad čak i najlošije jedinke prežive u iduću generaciju. Osim `survivalRate` parametra, ovaj operator ima i parametar `p` koji određuje parametar geometrijske distribucije.

3.10. Križanje

Križanje je proces kombiniranja gena dvaju roditelja s ciljem stvaranja potomaka koji od roditelja nasljeđuju dobra svojstva rješenja. Ovo usmjeravanje pretrage još se naziva eksploatacija, odnosno intenzifikacija zato što se trudimo iskoristiti dobra svojstva jednog rješenja za pronalazak idućeg boljeg rješenja, odnosno usmjeriti pretragu u smjeru baš tog rješenja. Osim pojedinih operatora križanja implementiran je i kompozitni operator križanja koji omogućuje izvršavanje svakog od operatora križanja djece s nekom vjerojatnosti kod svakog poziva kompozitnog križanja. U svrhu demonstracije, jedinke ćemo od sada prikazivati kao matrice nula i jedinica radi jednostavnosti, matrice kao takve zapravo ne postoje u implementaciji, ali rezultati operatora mogu se opisati unutar okvira takvih matrica. Prisjetimo se, retci u matrici predstavljaju pojedine trgovine dok stupci predstavljaju specifične nedjelje.

3.10.1. Križanje s k izmjena redaka

Jedan od načina na koji se dobar genetski materijal može prenijeti sa roditelja na potomke jest kopija, odnosno zamjena rasporeda pojedine trgovine kroz godinu. Ovu operaciju vrlo je jednostavno izvesti iz razloga što ona zasigurno ne mijenja zadovoljenost ograničenja – ako su ograničenja zadovoljena prije križanja tada će ograničenja biti zadovoljena i nakon križanja. Ovaj postupak može se ponoviti za k trgovina istovremeno, prikazano na slici 7, što ovaj operator križanja nudi kao postavku kroz svoje sučelje. Osim te postavke, u ovome operatoru nudimo i postavku za vjerojatnost križanja p koja omogućuje da ovaj operator baš roditelje, bez ikakvih izmjena, proslijedi operatoru mutacije.



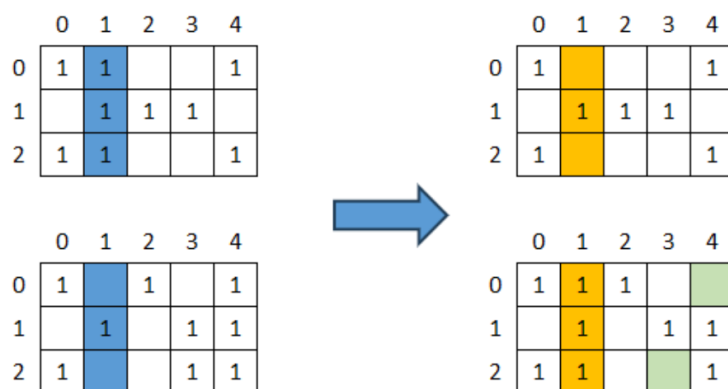
Slika 7 - dijagram križanja s k izmjena redaka

Implementirana je i varijanta ovog operatora koja umjesto fiksnog broja k , broj k uzorkuje iz geometrijske razdiobe zadane postavkom `geometricP`. Ta varijanta operatora i dalje nudi postavku `crossoverP` koja definira vjerojatnost križanja.

Osim te varijante, implementirana je i varijanta koja je slična takozvanom križanju s jednom točkom prekida (engl. `single point crossover`). Ova varijanta u svojoj implementaciji nigdje ne specificira parametar k , već odabire nasumičnu točku unutar liste te primjerice, sve trgovine prije točke uzima od prvog roditelja, a sve trgovine nakon točke uzima od drugog roditelja. Križanje s jednom točkom prekida intuitivno ne doprinosi kvalitetnom prijenosu gena zato što poredak trgovina unutar rješenja nije bitan, no ova varijanta operatora može se koristiti ako velike dijelove rješenja želimo prenijeti potomcima.

3.10.2. Križanje s k izmjena stupaca

Osim izmjenom redaka, odnosno rasporeda trgovina unutar rješenja, rješenja možemo križati i izmjenom stupaca odnosno izmjenom rasporeda za pojedine nedjelje. S obzirom da ćemo funkciju vrijednosti odnosno funkciju dobrote računati na temelju trgovina koje rade pojedine nedjelje, izmjena rasporeda za pojedinu nedjelju vrlo će dobro iskoristavati dobre rasporede za pojedine nedjelje. Mana ovog pristupa jest što ovaj operator po svojoj prirodi ne čuva stanje ograničenja rješenja već je nakon svakog križanja potomak potrebno „popraviti“. Popravak rješenja vrlo je zahtjevna procedura koja za mnogo ulaznih trgovina i mnogo ulaznih nedjelja može potrajati. Popravak rješenja vrši se na način da se za pojedinu trgovinu, radne nedjelje uklanjaju na nasumičan način ako ih je previše, odnosno radne nedjelje dodaju na nasumičan način ako ih je premalo. Ostatak rada operatora analogan je radu operatoru križanja s k izmjena redaka. Dijagram procedure za križanje s k izmjena stupaca nalazi se na slici 8. Za ovaj operator implementirane su identične varijante onima kod prijašnjeg operatora te u svojem sučelju nude identične postavke.



Slika 8 - dijagram križanja s k izmjena stupaca

3.11. Mutacija

Operator mutacije po svojoj je prirodi stohastički operator koji služi za istraživanje novih prostora rješenja (eksplozacija / diverzifikacija). Iz tog razloga, semantika iza ponašanja operatora mutacije nije potrebna kao što je slučaj kod operatora mutacije. Iz tog razloga implementiran je samo jedan operator mutacije, a to je standardni slučajni mutator na slici 9 koji ima parametre `numMutations` koji određuje broj mutacija i parametar `p` koji

predstavlja vjerojatnost mutacija. Operator radi na način da `numMutations` puta, za slučajnu trgovinu, slučajnu nedjelju iz modela zamijeni nekom slučajnom nedjeljom iz antimodela. Ova implementacija čuva stanje ograničenja i nakon mutacije ne zahtjeva nikakve „popravke“. Osim slučajnog operatora mutacije implementiran je i kompozitni operator mutacije koji može sadržavati više slučajnih operatora mutacije sa različitim intenzitetima koji se tada nad potomcima izvršavaju s različitim vjerojatnostima. Vjerojatno je slučaj da operatore mutacije s većim intenzitetom želimo izvršavati rjeđe.



Slika 9 - primjer mutacije nad jednim retkom

3.12. Funkcija dobrote

Prisjetimo se, cilj algoritma maksimizirati je pokrivenost područja s trgovinama koje rade za svaku nedjelju, a u isto vrijeme minimizirati kanibalizaciju među trgovinama kako bi svaka trgovina na kraju imala maksimalan profit s obzirom na utjecaj drugih trgovina u njejoj okolini. Kako bi strukturirano modelirali funkciju dobrote koja se može primijeniti na bilo kakav skup podataka funkciju dobrote računati ćemo u koracima na način da prvo definiramo vrijednost trgovine s obzirom na njenu okolinu, s obzirom na vrijednost trgovine modeliramo radijus utjecaja te trgovine na svoju okolinu i konačno na temelju izračunatih radijusa za sve trgovine u okolici izračunamo vrijednost funkcije dobrote. Cilj je, da u konačnici, funkcija dobrote bude glatka kako bi je algoritam mogao optimirati i da cijela procedura u konačnici dobro modelira očekivano ponašanje kupaca za neku nedjelju.

3.12.1. Funkcija vrijednosti

Uloga funkcije vrijednosti je definirati preslikavanje između ulaznog skupa podataka u metriku koja na neki način opisuje veličinu trgovine. Za funkciju vrijednosti idealno treba vrijediti pretpostavka da ukoliko trgovina A ima veću vrijednost funkcije vrijednosti od trgovine B, u izolaciji, trgovina A očekuje veći promet od trgovine B te nedjelje odnosno

očekuje se da će više ljudi posjetiti trgovinu A od trgovine B te nedjelje. Dobra funkcija vrijednosti dobro modelira taj odnos na način da veće razlike u vrijednosti vode to većih razlika u prometu te trgovine te nedjelje. Za primjer, na skupu podataka prikupljenom s javno dostupnih internetskih izvora za funkciju vrijednosti koristiti ćemo točno vrijednost polja `user_ratings_total`, odnosno broj recenzija za danu trgovinu. Na dobrom skupu podataka, funkcija vrijednosti treba se razlikovati iz nedjelje u nedjelju.

3.12.2. Funkcija za izračun radijusa

Radijus na kojem trgovina ima utjecaj modeliran je na temelju zakona gravitacije u maloprodaji kojega je opisao američki ekonomist William J. Reilly 1931. [5] Zakon je heuristika temeljena na Newtonovom gravitacijskom modelu kod kojeg umjesto mase tijela koristi mjeru za privlačnost (engl. *attractivness*) trgovine, koju mi u našem kontekstu nazivamo vrijednost.

Prvo se za svaku trgovinu računa njen bazični radijus koji se računa na temelju najvećeg dozvoljenog radijusa kao i odnosa vrijednosti te trgovine s obzirom na najveću trgovinu u skupu podataka. S obzirom da želimo preslikati vrijednost trgovine u njenu površinu, a ne nužno njen radijus, u formuli za omjer vrijednosti koristimo korijen. Bazični radijus predstavlja radijus te trgovine u izolaciji od ostalih trgovina, samo na temelju metrike vrijednosti V_i .

$$R_{base,i} = R_{max} \cdot \sqrt{\frac{V_i}{V_{max}}}$$

Commented [SD12]: UPDATE: Sad je jasno sto je V

Potom, modeliramo gravitacijski pritisak na trgovinu koji modelira koliko na trgovinu utječu trgovine u njejoj okolini. S uzorom na Newtonov gravitacijski model, pretpostavljamo da je pritisak proporcionalan relativnoj vrijednosti trgovine (masi m_j) i obrnuto proporcionalan kvadratu udaljenosti trgovina $d_{i,j}$. Pritisak možemo usporediti sa gravitacijskom silom u fizici. Udaljenost trgovina se s obzirom na njihove koordinate računa koristeći Haversine formulu.

$$P_i = \sum_{j \neq i} \frac{m_j}{d_{i,j}^2}$$

Commented [SD13]: UPDATE: Sad je jasno i sto su m i d

Konačno, na bazični radijus primjenjuje se osnovna, hiperbolička funkcija prigušenja pritiskom (engl. Decay function) kako bi se odredio konačni radijus trgovine.

$$R_{final,i} = \max(R_{min}, \frac{R_{base,i}}{1+\alpha \cdot P_i})$$

Parametar α određuje osjetljivost trgovine na pritisak. Ako je $\alpha \cdot P_i$ blizu nuli trgovina je u izolaciji i zadržava svoj bazični radijus. Ako je $\alpha \cdot P_i$ velik, konačni radijus proporcionalno je manji.

U najboljem slučaju pritisak na trgovinu P računa se na temelju samo trgovina koje rade te nedjelje. Međutim, računanje konačnog radijusa za svako rješenje mnogo bi usporilo algoritam i kao rezultat dalo rješenja koja je zahtjevno vizualno usporediti. S obzirom da je ove brojeve već sada zahtjevno intuitivno tumačiti, odlučili smo pritisak na svaku trgovinu računati unaprijed i konačni radijus tumačiti samo kao područje utjecaja trgovine. Nadogradnja algoritma, uz vjerodostojnije podatke, bila bi tada računati konačni radijus za svaku drugačiju konfiguraciju nedjelje.

3.12.3. Kombinirana funkcija dobrote

Konačno ćemo izračunate radijuse utjecaja primijeniti na način da nad svakom trgovinom definiramo područje utjecaja. Područje utjecaja biti će kvadrat s središtem na koordinatama trgovine, a koji ima stranicu duljine $R/2$. Područje utjecaja intuitivnije bi bilo opisati krugovima, međutim s obzirom da ćemo nad navedenim oblicima raditi operacije presjeka i unije, te operacije mnogo je jednostavnije i brže izvesti ako su ti oblici isključivo kvadrati. Osim toga, u naše svrhe kvadrat dovoljno dobro opisuje područje utjecaja. Iscrtavanjem kvadrata dobivenih koristeći prethodno opisani postupak, za trgovine koje neke nedjelje u rasporedu rade, dobit ćemo područje utjecaja s preklapanjima prikazano na slici 10 plavom bojom. Kako bi u obzir uzeli kriterij pokrivenosti, našoj funkciji dobrote dodati ćemo površinu unije iscrtanih kvadrata prikazano na slici 10 crvenom bojom. U svrhu „kažnjavanja“ u okviru kriterija kanibalizacije od funkcije dobrote oduzimati ćemo svako područje na kojem barem dvije trgovine imaju preklapanje u području utjecaja, takvo područje prikazano je na slici 10 zelenom bojom. Naša konačna funkcija dobrote biti će jednaka površini žutog lika na slici 10.



Slika 10 - dijelovi konačne funkcije dobrote

Bitno je napomenuti kako algoritam rezultate funkcije dobrote tumači na način da veći iznos funkcije dobrote predstavlja bolju jedinku, to bi trebalo biti i jasno iz samog naziva funkcije dobrote. Pretpostavka je da će ovaj model, zajedno sa modelom za funkciju vrijednosti i izračun radijusa dobro modelirati volumen prodaje s obzirom na trgovine koje rade specifične nedjelje. Analogija koja se iz prethodnog primjera može izvući jest da površina predstavlja dostupan novac, odnosno mušterije na nekom području pa tada područje unije predstavlja ukupno količinu građana koji su pokriveni rasporedom dok područje presjeka predstavlja količinu građana kod kojih će doći do kanibalizacije radi preklapanja područja na kojem djeluju trgovine.

3.13. Paralelizacija algoritma

Implementacijom algoritma u Javi omogućili smo jednostavno proširenje algoritma paralelizacijom. Genetski algoritmi po svojoj su prirodi pogodni za paralelizaciju jer spadaju u klasu algoritama koji se još nazivaju trivijalno paralelnima (engl. embarrassingly parallel). To znači da se algoritam može raspodijeliti na više procesorskih jezgri ili dretvi uz vrlo malo truda i gotovo bez potrebe za sinkronizacijom i komunikacijom između dretvi. Paralelizacija se kod genetskih algoritama najčešće uvodi kod računanja funkcije dobrote zato što je baš funkcija dobrote najčešće usko grlo izvršavanja algoritma (engl. bottleneck). Kada je to slučaj, genetski algoritam trivijalno se paralelizira tako da se određeni broj izračuna funkcije dobrote podijeli između više računskih jedinica s ciljem linearnog ubrzanja računa.

Međutim, ispada da naša implementacija funkcije dobrote ipak nije usko grlo kod izvođenja. Naime, korištenjem podatka da su svi objekti kod izračuna presjeka odnosno unije točno kvadrati proizvoljnih radijusa uspjeli smo dobiti ubrzanje od 200 puta u odnosu na implementaciju koja tu pretpostavku ne uzima u obzir. Kao rezultat ubrzanja, evaluacija funkcije dobrote za 10000 kromosoma traje samo približno 300 ms. Implementacija trivijalne paralelizacije kod izračuna funkcije dobrote imala bi smisla samo kod vrlo velikih

populacija, koje ćemo izbjegavati iz razloga što se kod velike populacije pretraživanje više zasniva na gruboj pretrazi ili pak konvergira prebrzo radi intenzivnog elitizma.

Umjesto trivijalne paralelizacije, iskoristiti ćemo prije opisani princip razdvajanja velikog problema na manje potprobleme koje ćemo svakog zasebno rješavati genetskim algoritmom. Naime, po uzoru na stablastu strukturu sa Slike 5 , sve čvorove na istoj razini stabla rješavati ćemo istovremeno, odnosno, svaki čvor će se rješavati u zasebnoj dretvi.

4. Infrastruktura za prezentaciju i automatsku optimizaciju hiperparametara

Iako je srž rješavanja problema raspoređivanja radnih nedjelja sadržana u matematičkom modelu i implementaciji genetskog algoritma, takav sustav u svom izvornom obliku (kao skripta koje se izvršava u terminalu) nije prikladan za krajnje korisnike. Ipak, algoritam rješava problem iz stvarnog svijeta pa bi prikladno bilo kada bi se on i mogao koristiti u stvarnom svijetu. Ciljani kupci ovakvog sustava bili bi voditelji maloprodajnih lanaca i trgovina koji najčešće nemaju napredno informatičko odnosno programersko predznanje. Njima je potrebno interaktivno sučelje koje predstavlja alat za vizualizaciju, postavljanje i prikaz rezultata problema. S obzirom na to da se naš problem uvelike oslanja na geografske podatke i geometrijske likove, čitanje optimalnog rasporeda iz tekstualnih ili matričnih zapisa je neintuitivno.

Nadalje, genetski algoritam kroz svoje sučelje definira niz hiperparametara kao što su veličina populacije, vjerojatnost mutacije, broj generacija koji utječu na brzinu i kvalitetu konvergencije. Implementaciju interaktivnog sučelja iskoristiti ćemo i u svrhu optimizacije prethodno navedenih hiperparametara.

4.1. Korištene tehnologije

Sučelje smo odlučili implementirati kao web-aplikaciju u klijentsko-poslužiteljskoj arhitekturi. Druga opcija za implementaciju bilo je implementacija nativnog sučelja, međutim zbog prijašnjeg iskustva implementacije web-aplikacija i složenosti sustava činilo se primjereno sustav ipak razdvojiti na komponente i koristiti baš one tehnologije s kojima smo otprije upoznati.

Sam algoritam implementiran je u nativnoj Javi 21 iz razloga što Java uz brzinu usporedivo s C++-om, ima uredan način implementacije objektno orijentiranim programiranjem koje je pogodno za implementaciju generičkog algoritma opisanog u prethodnim poglavljima.

Poslužiteljska strana implementirana je u Python koristeći okvir Flask. Flask je odabran radi svoje jednostavnosti implementacije kao i jednostavnoj proceduri puštanja u produkciju (engl. deployment). Poslužitelj sa korisničkog sučelja prima podatke kao i postavke za algoritam i postavke problema, konstruira potrebni zadatak koji izvršava pozivom

kompajliranog algoritma u Javi i korisničkom sučelju vraća rezultate i kao i ažurne informacije o toku algoritma.

Za klijentsku stranu, odnosno korisničko sučelje korišten je okvir NextJS. NextJS nije najbolji izbor gledajući performanse zato što je ideja NextJS-a da se klijentska i poslužiteljska strana pišu u „istom“ izvornom kodu. Unatoč tome, NextJS koristimo iz razloga što nudi dobru okolinu korištenje React tehnologije dok u isto vrijeme nudi jednostavno usmjeravanje po stranicama (engl. routing) koje je inače nezgodno implementirati koristeći nativni React.

4.2. Poslužitelj

Poslužitelj je implementiran na način da je pogodan za korištenje i od strane korisničkog sučelja i od strane programske skripte s obzirom da isti poslužitelj planiramo koristiti i za orkestraciju kod optimizacije hiperparametara algoritma.

Pristupne točke (engl. endpoint) poslužitelja implementirane su koristeći dobre prakse RESTful pristupa, međutim s obzirom da poslužitelj u isto vrijeme služi za udaljene pozive procedura (engl. remote procedure call) neke pristupne točke odstupaju od tog pristupa. Neki dijelovi sustava koriste i tehnologiju WebSocket kako bi korisničko sučelje efikasnom komunikacijom bilo informirano o događanjima kod izvršavanja algoritma.

Iako ćemo poslužitelj i sučelje generalno koristiti kao interni alat, poslužitelj je potrebno zaštititi autentifikacijom iz razloga što će poslužitelj i sučelje ipak biti izloženi na internetu na kojem djeluju maliciozni akteri. Autentifikacija lozinkom implementirana je preporučenim kriptografskim pristupom koristeći najkorištenije algoritme i procedure kao što je BCrypt. Kreaciju računa vrši glavni administrator koji za to treba imati direktan pristup poslužitelju koristeći ssh.

Poslužitelj prijavljenom korisniku omogućuje stvaranje i uređivanje svakog izvršavanja algoritma odnosno takozvanog „zadatka“. Postavljanje zadatka uključuje učitavanje podataka, postavljanje ograničenja i postavljanje postavki algoritma. Poslužitelj za svakog jedinstvenog korisnika čuva mapu sa svim njegovim zadacima i generiranim rezultatima odnosno dnevnikom izvršavanja. Perzistentnost podataka vrši se isključivo u okviru operacijskog sustava, bez baze podataka s ciljem smanjenja tehnološke infrastrukture.

Osim stvaranja i uređivanja zadatka, poslužitelj korisniku nudi kontrolu i nadzor izvršavanja pojedinog zadatka na način da ima mogućnost, koristeći WebSocket, komunicirati dnevnik izvršavanja i nudi naredbe kojima se određeni zadatak može pokrenuti odnosno prisilno zaustaviti. Ovo je sve ostvareno koristeći Python okvir subprocess koji omogućuje stvaranje i zaustavljanje novih procesa. Konkretno, poslužitelj orkestrira pokrenute jar programe koji izvršavaju genetski algoritam.

Nakon izračuna rezultata, poslužitelj nudi i sučelje prema dobivenim rezultatima. Na ovaj način, rezultati su generirani i interpretirani u istom čvoru čime se izbjegavaju moguće razlike između vizualnog sučelja i poslužitelja.

4.3. Postavljanje aplikacije

Radi jednostavnosti korištenja, poslužitelj za zadatke kao i poslužitelj korisničkog sučelja izloženi su na internetu. Za postavljanje obje su komponente zapakirane u Docker kontejnere kako bi njihovo postavljanje bilo nezavisno s obzirom na računalo domaćina.

Kontejner poslužitelja korisničkog sučelja konstruiran je na temelju minimalne Node slike (engl. image), te se datoteke korisničkog sučelja jednostavno poslužuju koristeći Node poslužitelj.

Konstrukcija kontejnera poslužitelja za zadatke nešto je zahtjevnija s obzirom da smo odlučili zajedno s Flask poslužiteljem isporučivati i izgrađenu (engl. build) jar datoteku koja se koristi za pokretanje algoritma. S obzirom na to, izgradnja kontejnera za poslužitelj sastoji se od nekoliko koraka. Prvi korak je izgradnja jar datoteke na temelju izvornog koda algoritma. U idućem koraku izgrađena datoteke algoritma i java datoteke spremaju se na lokaciju unutar slike. Za komponente napisane u programskom jeziku Python, potrebno je instalirati i sve zahtjeve koje one imaju. Konačno, za rad poslužitelja koristi se gunicorn poslužitelj koji unutar kontejnera prima i odgovara na HTTP zahtjeve koje dobiva.

Konstruirani kontejneri učitavaju se na privatni račun na platformi dockerhub odakle ih se može preuzeti s udaljenog računala. Nakon učitavanja, moguće je pozvati pripremljenu bash skriptu s udaljenog poslužitelja kako bi se naši poslužitelji za zadatke i korisničko sučelje preuzeli i pokrenuli.

Privatno virtualno udaljeno računalo (engl. virtual private server) unajmljeno je na platformi Hetzner te se osim za ovaj projekt koristi i za neke druge. Na virtualnom računalu upogonjen

je nginx posrednički poslužitelj (engl. proxy) koji zahtjeve, s obzirom na domenu zahtjeva, prosljeđuje ispravnom pokrenutom Docker kontejneru.

Procedura za postavljanje vrlo je složena te se sastoji od puno koraka i naredbi koje je potrebno pokrenuti u naredbenom retku lokalnog računala kao i udaljenog računala. Kako bi se postavljanje aplikacije nakon malih koraka automatiziralo, implementirani su CI/CD tokovi koji ove korake, na strukturirani način, redom izvršavaju. Repozitorij sa svim izvornim kodom održavan je na platformi GitHub te se za CI/CD procedure koriste njihovi GitHub Actions koji neprimjetno pokreću procedure za postavljanje nakon svakog commit-a koji to traži. U *commit* poruci moguće je specificirati koji dio sustava je potrebno ponovno postaviti (poslužitelj za zadatke, poslužitelj za sučelje ili oboje) kako bi se smanjila potrošnja minuta koje su dostupne na GitHub Actions-u bez plaćanja.

4.4. Korisničko sučelje

Korisničko sučelje dizajnirano je s ciljem da se običan korisnik, uz malo treninga u sustavu uspije snaći. Sučelje je dizajnirano na način da se zadatak za optimizaciju stvara u koracima, od unosa podataka, do postavljanja postavki algoritma pa konačno i sučelja za izvršavanje optimizacije i pregled dobivenih rezultata.

4.4.1. Popis zadataka

Nakon autentifikacije, korisnik se prvo navigira na stranicu /jobs (slika 11) gdje ga očekuje popis svih zadataka koje je on kreirao. U jednostavnoj tablici, moguće je vidjeti opis zadatka, datum i vrijeme kada je stvoren te konačno status zadatka koji može biti

1. Nije inicijaliziran (engl. uninitialized) – izvršavanje optimizacije nikada nije bilo pokrenuto
2. Izvršava se (engl. running) – izvršavanje optimizacije trenutno je u tijeku
3. Greška (engl. error) – desila se greška tijekom optimizacije ili je optimizacija zaustavljena
4. Završeno (eng. complete) – optimizacija je uspješno završila i rezultati su spremni za pregled

Na tablici sa Slike 11 sa zadacima korisnik ima mogućnost brisati zadatke kao i stvarati nove zadatke.

The screenshot shows the 'Jobs' page of the JetPans application. At the top, there is a navigation bar with buttons for 'CONTROL', 'Working Sundays', 'Server' (sundays.jetpans.com), 'User' (jetpans), 'Edit', 'Session' (Authenticated), and 'Logout'. Below this, there are buttons for 'Jobs' and 'API Docs'. The main content area is titled 'Jobs' and features a search bar for 'Description (optional)' and a 'Create Job' button. A table lists the following jobs:

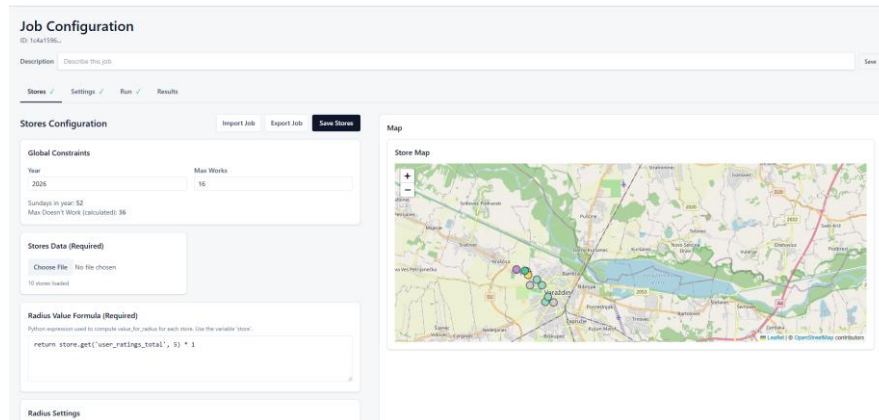
Job ID	Description	Created	Status	Actions
1c4a1596...	-	5/14/2026 at 7:11:35 PM	Complete	Delete
2a97cdf...	My favorite	5/14/2026 at 8:39:22 PM	Uninitialized	Delete
61e85067...	-	5/16/2026 at 4:42:07 PM	Uninitialized	Delete
76d6da56...	-	5/12/2026 at 1:39:30 PM	Complete	Delete
d9e1e7e4...	-	5/12/2026 at 1:13:16 PM	Complete	Delete

Slika 11 - stranica /jobs korisničkog sučelja

Nakon klika na tipku za stvaranje novog zadatka, u tablici se pojavljuje novi zadatak sa statusom „Uninitialised“. Klikom na neki zadatak otvara se sučelje sa skupom kartica (engl. tab) koji korisniku omogućuju strukturirano i postepeno definiranje zadatka. Kartice koje su potom dostupne su

1. Stores – kartica za stvaranje, odnosno definiciju novog skupa podataka
2. Settings – kartica za postavke parametara i hiperparametara genetskog algoritma, odnosno algoritma za grupiranje
3. Run – kartica u kojoj se pokreće zadatak i gdje se može vidjeti dnevnik izvršavanja
4. Results – kartica za pregled rezultata optimizacije

4.4.2. Stvaranje novog skupa podataka (Stores kartica)



Slika 12 - kartica za stvaranje novog skupa podataka

Kartica za stvaranje novog skupa podataka (slika 12) nudi sučelje za definiciju problema koji ćemo rješavati genetskim algoritmom. Kartica se sastoji od polja za definiciju općenitih ograničenja kao što je broj radnih nedjelja u nekoj godini i polja za učitavanje datoteke s podacima o trgovinama. Datoteka s podacima o trgovinama mora biti ispravnog formata kako bi se mogla učitati. Pretpostavka je da neki sustav koji stvara podatke ima implementiranu skriptu koja može podatke ispisati točno u našem, traženom formatu. Učitane trgovine moguće je pregledati na karti kao i u tablici u kojoj se nalazi popis svih učitanih trgovina. Klikom na trgovinu u tablici moguće je postaviti ograničenja kao na slici 13 na nedjelje kada trgovina mora raditi odnosno kada trgovina ne smije raditi



Slika 13 - sučelje za uređivanje ograničenja

Osim učitavanja trgovina i postavljanje ograničenja, u sučelju za stvaranje novog skupa podataka nalaze se i postavke za parametre Reillyevog gravitacijskog modela kao i formula funkcije vrijednosti koja se izvršava na poslužitelju nakon klika za spremanje trgovina. Parametre gravitacijskog modela kao i formulu funkcije vrijednosti ne smatramo hiperparametrima algoritma, već samo parametrima skupa podataka. Razlog tome je što o navedenim parametrima ovisi funkcija dobrote te na ovaj način oni služe isključivo za postavke modela koji optimiramo, a postavke modela trebaju uključivati domensko iskustvo što ih bitno razlikuje od drugih hiperparametara i parametara algoritma.

4.4.3. Definiranje hiperparametara algoritma (Settings kartica)

The screenshot shows a settings card for a genetic algorithm. It includes the following fields and options:

- Num Threads:** 4
- Deterministic:** ☒
- Mutator:** CompositeMutator (dropdown menu)
- P:** 1
- Weights are set per child below (default 1.0):**
- Children:** Composite Children (button), + Add child to composite (link)
- Crossover:** CompositeCrossover (dropdown menu)
- P:** 1
- Weights are set per child below (default 1.0):**
- Children:** Composite Children (button), + Add child to composite (link)
- Selection:** TournamentSelection (dropdown menu)
- tournamentSize:** 3
- Fitness:** FastIntersectUnionFitness (dropdown menu)
- No additional parameters**

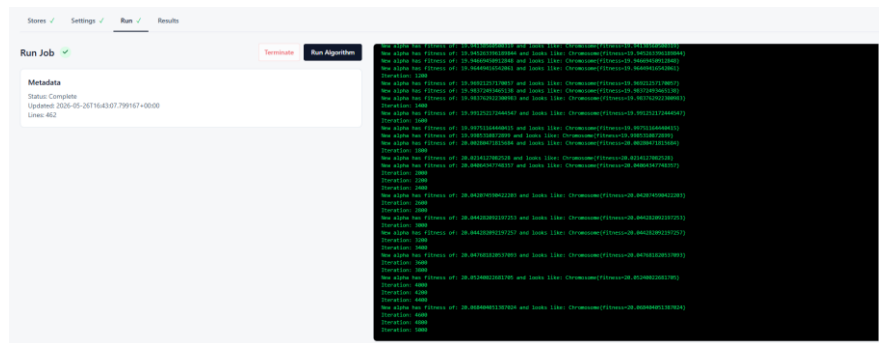
Slika 14 - dio kartice za postavke

U kartici za postavke (slika 14) definiraju se postavke genetskog algoritma kao i algoritma grupiranja. Svaki ponuđeni genetski operator može se odabrati iz izbornika. Zanimljiva je činjenica da je omogućeno i definiranje složenih kompozitnih operatora kod implementacije čega je vrlo dobra došla pomoć alata umjetne inteligencije zato što je strukture podataka za navedene operatore potrebno definirati u svakoj komponenti sustava od korisničkog sučelja, preko poslužitelja zadataka pa čak i u konkretnoj implementaciji algoritma gdje je potrebno JSON format prevesti u definiciju instance klase. Baš za ovakve zadatke su alati umjetne

inteligencije idealno rješenje čime omogućuju vrlo jednostavno proširenje cijelog sustava novim genetskim operatorom ili novom postavkom unutar algoritma. O konkretnim postavkama i mogućnostima genetskih operatora više ćemo pisati u poglavlju o hiperparametrima gdje ćemo trebati točno definirati sve postavke i domene u kojima ćemo pretraživati.

4.4.4. Pokretanje algoritma (Run kartica)

Kartica za pokretanje algoritma (slika 15) sastoji se od tipki za pokretanje odnosno prisilno zaustavljanje optimizacije genetskim algoritmom. Klikom na tipku za pokretanje poslužitelj stvara novi proces koji koristi prikladan broj dretvi koje su dostupne na sustavu. Klik na tipku za prisilno zaustavljanje pak isti proces zaustavlja SIGKILL signalom. Na kartici se još nalazi i statusna traka sa podacima o najnedavnijem izvršavanju optimizacije kao i prozor koji sa poslužitelja uživo dohvaća dnevnik izvršavanja koristeći protokol WebSocket.

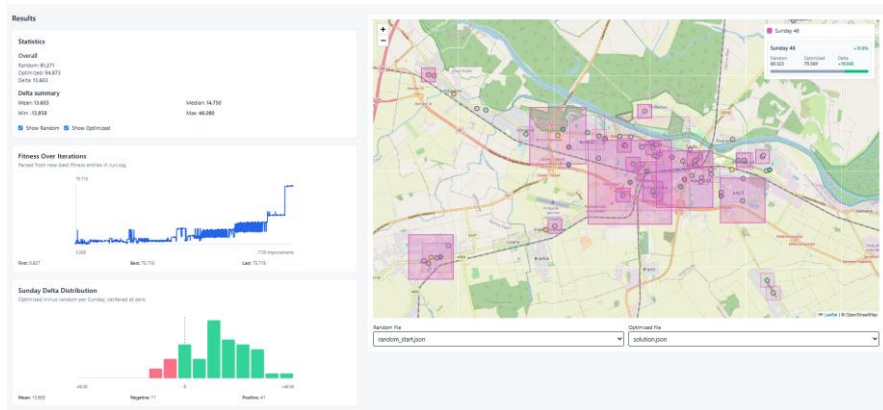


Slika 15 - kartica za pokretanje algoritma

4.4.5. Pregled rezultat (Results kartica)

Na kartici s rezultatima (slika 16) moguć je direktan uvid u izgled funkcije dobrote za pojedine nedjelje. Na desnoj strani kartice nalazi se karta s prikazanim trgovinama u skupu podataka. U izborniku na karti moguće je odabrati nedjelju ili više njih za koje želimo vidjeti radno vrijeme na tome području. Na karti se tada prikazuju kvadrati koji predstavljaju površine iz izračuna funkcije dobrote, a u legendi karte za odabrane nedjelje vidi se promjena funkcije dobrote s obzirom na neki nasumični raspored, bila ona pozitivna (poboljšanje) ili negativna (pogoršanje). Korisnik može odabrati želi li gledati raspored za optimalno rješenje, nasumično rješenje ili oba istovremeno. Kvalitetu rješenja teško je intuitivno

razumjeti i prihvatiti, no usporedba s nasumičnim rješenjem ispala je najboljom intuitivnom usporedbom s uvidom u kvalitetu rada algoritma.



Slika 16 - kartica za prikaz rezultata

Commented [SD14]: UPDATE: Nova results tab slika

5. Optimizacija hiperparametara i podrezultati

Kvaliteta rezultata algoritma uvelike ovisi o odaberu hiperparametara. S obzirom na dimenziju prostora hiperparametara koja se u našem sastoji od 34 varijable pretraga koja isprobava sve moguće kombinacije (engl. grid search) ne dolazi u obzir. Umjesto grube pretrage, odlučili smo koristiti gotovu implementaciju Bayesovske optimizacije iz knjižnice `scikit-optimize`. Bayesovska optimizacija industrijski je i akademski standard za optimizaciju hiperparametara. Za razliku od slučajne pretrage, Bayesovska optimizacija koristi prošle evaluacije kod modeliranja probabilističkog modela koji se koristi za predviđanje još neisprobanih kombinacija hiperparametara. Probabilistički model koji se u pozadini koristi jest Gaussov proces koji se pokazao kao vrlo dobar model za skupove podataka s malim brojem uzoraka. Za idući isprobani skup hiperparametara nasumično se koristi jedna od tri akvizicijske metode koje temeljem različitih uvjeta uzorkuju prostor hiperparametara. Očekivano je da algoritam nakon nekog vremena konvergira u neki lokalni optimum sa hiperparametrima koji daju dobre rezultate na testnom skupu trgovina i ograničenja.

Radi ravnopravnosti između uzoraka hiperparametara, vrijeme izvođenja pojedinog genetskog algoritma je fiksirano kako prednost, zbog svoje robustnosti, ne bi imali skupovi hiperparametara koji koriste veće populacije odnosno izvršavaju se u više iteracija. S obzirom na to da koristimo više vrsti genetskih operatora za svaki genetski operator prostor hiperparametara modelirati ćemo na način da svaku podvrstu genetskog operatora stavimo u jedan kompozitni operator u nadi da će težine kompozitnog operatora konvergirati prema onome najboljem, odnosno najboljoj distribuciji za odabir konkretnog operatora. Ovu logiku primjenjujemo na operatore mutacije i križanja, dok isto nema smisla raditi za operatore selekcije i eliminacije pa ćemo kod tih operatora „žrtvovati“ nekoliko hiperparametara koji, ovisno o konkretnom skupu hiperparametara neće utjecati na optimizacijski postupak. Popis svih hiperparametara zajedno s njihovim opisima i prostorom pretrage nalazi se u Tablici 1.

Radi dobrog pokrivanja prostora problema, za evaluaciju koristiti ćemo nekoliko instanci skupova podataka tako da bi rezultatni skup hiperparametara trebao dobro funkcionirati na različitim skupovima trgovina.

Rezultate uzoraka hiperparametara zajedno sa njihovim konačnim, najboljim, vrijednostima funkcije dobrote spremati ćemo te na kraju napraviti analizu pojedinih hiperparametara koji bi najviše mogli utjecati na kvalitetu rezultata algoritma.

Tablica 1 - popis hiperparametara i prostora pretrage

Varijabla	Opis	Prostor pretrage	Optimalna vrijednost
MAX_CLUSTER_DISTANCE	Najveća najmanja udaljenost između nove točke grupe i neke točke u grupi [km]	$x \in [1, 12], x \in R$	1
MAX_CLUSTER_JOIN_DISTANCE	Najveća udaljenost iznad koje nema smisla spajati grupe [km]	$x \in [2, 20], x \in R$	15
populationSize	Veličina populacije u jednom genetskom algoritmu	$x \in \{50, 200, 500, 1000, 5000\}$	1000
Selection type	Koji operator selekcije koristiti	„TournamentSelection“, „RankSelection“	Tournament Selection
Selection_tournamentSize	Veličina turnira u turnirskoj selekciji	$x \in [2, 6], x \in N$	5
Elimination type	Koje operator eliminacije koristiti	„EliteEliminator“, „EliteGeometricEliminator“	Elite Eliminator
Eliminator_elitism	Broj jedinki koje preživljavaju eliminaciju u	$x \in [1, 20], x \in N$	5

Commented [SB15]: Izbrisan generations parametar kojeg nema smisla optimirati. (DODANI optimalni parametri na ostale parametre)

	elitističkom eliminatoru		
Eliminator_ survivalRate	Udio jedinki koje preživljavaju u geometrijskom eliminatoru.	$x \in [0.05, 0.5], x \in R$	NaN (0.128)
Eliminator_p	Vjerojatnost korištena kod geometrijske distribucije u geometrijskom eliminatoru	$x \in [0.1, 0.95], x \in R$	NaN (0.1)
Mutator_child1_ numMutations	Broj mutacija prvog mutatora	$x \in [1, 10], x \in N$	3
Mutator_child2_ numMutations	Broj mutacija drugog mutatora	$x \in [5, 20], x \in N$	10
Mutator_child1_p	Vjerojatnost mutacije prvog mutatora	$x \in [0.1, 0.9], x \in R$	0.64
Mutator_child2_p	Vjerojatnost mutacije drugog mutatora	$x \in [0.1, 0.9], x \in R$	0.1
Mutator_child1_ weight	Vjerojatnost odabira prvog mutatora	$x \in [0, 10], x \in R$	0.48
Mutator_child2_ weight	Vjerojatnost odabira drugog mutatora	$x \in [0, 10], x \in R$	0.43
crossover_child1_ geoP	Parametar geometrijske distribucije geometrijskog križanja po stupcima	$x \in [0.1, 0.8], x \in R$	0.50

crossover_child1_ crossoverProb	Vjerojatnost križanja prvog križanja	$x \in [0.1, 0.95], x \in R$	0.91
crossover_child1_ weight	Vjerojatnost odabira prvog križanja	$x \in [0.1, 1], x \in R$	1.00
crossover_child2_ geoP	Parametar geometrijske distribucije geometrijskog križanja po retcima	$x \in [0.1, 0.95], x \in R$	0.66
crossover_child2_ crossoverProb	Vjerojatnost križanja drugog križanja	$x \in [0.1, 0.95], x \in R$	0.20
crossover_child2_ weight	Vjerojatnost odabira drugog križanja	$x \in [0.1, 1], x \in R$	0.75
crossover_child3_p	Vjerojatnost križanja operatora križanja s jednom točkom prekida	$x \in [0.1, 0.9], x \in R$	0.42
crossover_child3_ weight	Vjerojatnost odabira trećeg križanja	$x \in [0.1, 1], x \in R$	0.72
crossover_child4_k	Parametar k operatora križanja s K izmjena redaka	$x \in [2, 8], x \in N$	6
crossover_child4_p	Vjerojatnost četvrtog križanja	$x \in [0.1, 0.95], x \in R$	0.21
crossover_child4_ weight	Vjerojatnost odabira četvrtog križanja	$x \in [0.1, 1], x \in R$	0.41

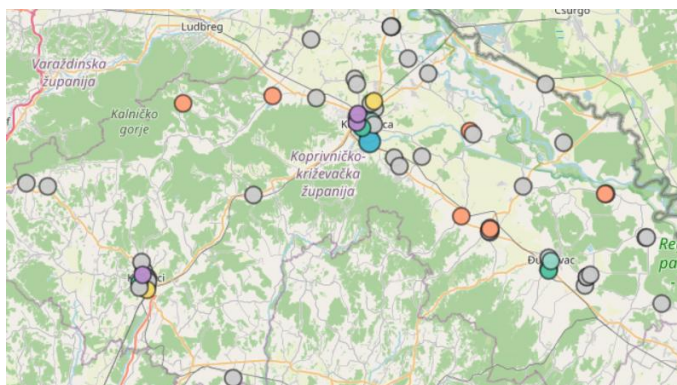
crossover_child5_k	Parametar k operatora križanja s K izmjena stupaca	$x \in [2, 8], x \in \mathbb{N}$	4
crossover_child5_p	Vjerojatnost petog križanja	$x \in [0.1, 0.95], x \in \mathbb{R}$	0.9
crossover_child5_weight	Vjerojatnost odabira petog križanja	$x \in [0.1, 1], x \in \mathbb{R}$	0.261

5.1. Skupovi podataka

Commented [SD16]: UPDATE: Skupovi podataka za optimizaciju hiperparametara

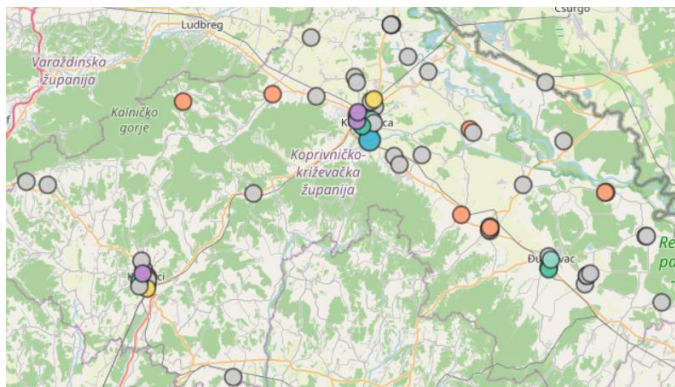
Za optimizaciju hiperparametara potrebno je odabrati skupove podataka koji dobro predstavljaju općeniti problem kako bi prilagođeni hiperparametri bili pogodni za bilo kakav ulazni skup trgovina i ograničenja. Skupove smo birali temeljem nekih njihovih značajki dok smo ograničenja za skupove podataka generirali nasumično.

Za prvi skup podataka odabrali smo sve trgovine koprivničko-križevačke županije koji se sastoji od 70 trgovina. Cilj tog skupa podataka bilo je imati skup podataka u kojem vrlo malo trgovina zapravo interagira kako bi algoritam sa optimalnim parametrima dobro radio na skupovima podataka kod kojih spajanje trgovina ima malen utjecaj i kod kojih je zapravo bitno optimizirati samo manje pod-skupove cijelog problema. Skup podataka trgovina u koprivničko-križevačkoj županiji prikazan je na slici 17.



Slika 17 – skup podataka koprivničko-križevačke županije

Za druga skup podataka odabrane su sve trgovine u Rijeci. Cilj ovog skupa podataka bilo je imati skup podataka u kojem svaka trgovina interagira sa velikim brojem drugih trgovina. Hiperparametri optimalni za navedeni problem dobro rade na skupovima podataka kod kojih spajanje trgovina i rješavanje pod-problema daje slabo informirane rezultate i kod kojih su kasniji koraci kod spajanja ključniji. Skup podataka trgovina u Rijeci prikazan je na slici 18.



Slika 18 - skup podataka u Rijeci

5.2. Rezultati optimizacije hiperparametara

Općeniti hiperparametri

Koristeći analizu parametara veličine populacije, broja generacija i broja novih kromosoma koji se dodaju svaku generaciju iz u Privitku 1 za konačan skup hiperparametara odabrati ćemo veličinu populacije od 1000 jedinki, sa 5000 generacija i 30 novih kromosoma u svakoj generaciji. Na dijagramima su baš ovi hiperparametri rezultirali s puno više kvalitetnih optimizacija nego onih loše kvalitetnih.

Također koristeći dijagrame iz Privitka 1, za parametre algoritma grupiranja odabrati ćemo najmanjih, 1 kilometar za najveću udaljenost između trgovina u početnim grupama dok ćemo za najveću udaljenost kod spajanja koristiti 15 kilometara. Udio optimizacija koje se nalaze u najboljoj četvrtini i koje koriste baš te parametre značajno je veći od onih optimizacija koje koriste drastično različite parametre od navedenih. Ovo prikazuje kako algoritam u početnim grupama preferira imati trgovine koje su što bliže, dok komponente njihove interakcije preferira optimirati u kasnijim iteracijama.

Commented [SB17]: UPDATE: Rezultati optimizacije HPO i odabir konacnih hiperparametara

Selekcija

Promatrajući kutijasti dijagram (engl. box plot) u Privitku 2 koji prikazuje distribuciju funkcije dobrote s obzirom na to koja vrsta selekcije je korištena zaključujemo kako nema značajne razlike između korištenog operatora. S obzirom na nešto veći medijan i maksimum i činjenicu da je češće korištena u genetskim algoritmima, za konačan skup hiperparametara odabrali smo turnirsku selekciju. Veličinu turnira postaviti ćemo na pet s obzirom na dijagram iz Privitka 1 koji prikazuje udio optimizacija koje su koristile taj hiperparametar i koje su završile u najboljoj četvrtini evaluacija (bolji od 75. kvantila) naspram onih koje su završile u najgoroj četvrtini evaluacija (lošiji od 25. kvantila).

Eliminacija

Prema kutijastom dijagramu u Privitku 3 koji prikazuje distribuciju funkcije dobrote s obzirom na to koja vrsta selekcije je korištena zaključiti ćemo i kako nema značajne razlike između korištenih operatora eliminacije. U konačni skup hiperparametara arbitrarno ćemo odabrati elitističku eliminaciju sa fiksnim parametrom elitizma od 5.

Križanje

Operacija križanja u konačnoj se optimizaciji hiperparametara sastojala od velikog broja pojedinačnih hiperparametara pa je gotovo nemoguće objektivno zaključiti najbolje parametre za svaki. Za operator križanja zanimljiv je dijagram iz Privitka 4 koji prikazuje težinsku distribuciju težina određenih implementacija težinski izračunatu s obzirom na vrijednost funkcije dobrote. Cilj dijagrama je pokušati numerički staviti u omjer učestalost korištenja operatora sa konačnom funkcijom dobrote. Dijagram prikazuje očekivanu razdiobu – geometrijski operatori križanja imaju veće vrijednosti pri čemu onaj koji djeluje nad pojedinim nedjeljama ima malu prednost. Za konačan skup hiperparametara koristiti ćemo dobivenu distribuciju dok ćemo za parametre pojedinih operatora jednostavno koristiti one parametre iz najbolje optimizacije.

Mutacija

Iz dijagrama za operatore mutacije, na našem skupu mjerenja nemoguće je donijeti čvrste zaključke. Dijagrami koji prikazuju distribucije dobrote naspram parametara vrlo su šumoviti i iz njih se može zaključiti samo da parametri operatora mutacije nisu ključni za kvalitetnu optimizaciju rasporeda. Za konačan skup hiperparametara jednostavno ćemo koristiti one parametre iz najbolje optimizacije.

6. Rezultati

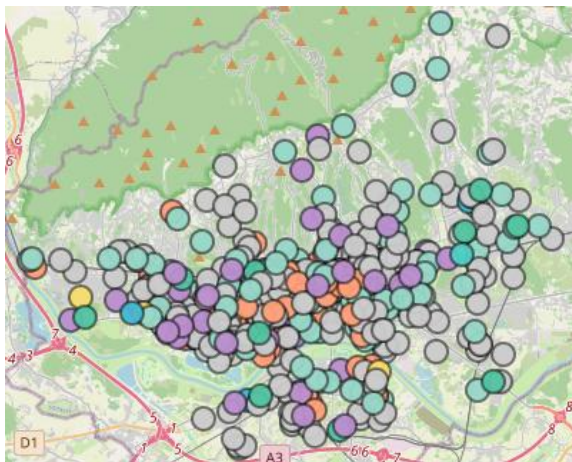
Commented [SD18]: UPDATE: Rezultati

6.1. Skupovi podataka

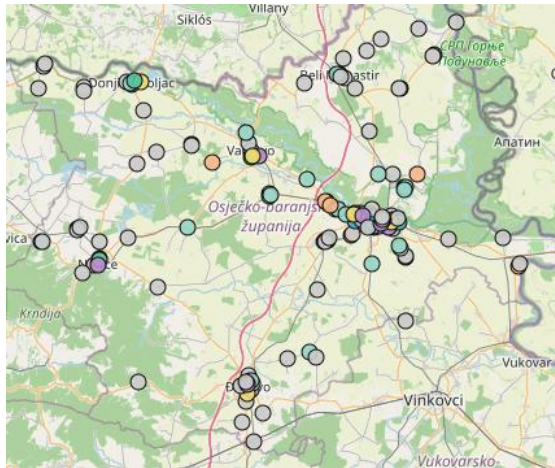
Analogno skupovima podataka korištenima za optimizaciju hiperparametara, za analizu konačnih rezultata odabrati ćemo skupove podataka koji imaju slična svojstva, ali u svrhu konačnih rezultata sastoje se od više trgovina.

Kao pandan skupu podataka u Rijeci koristiti ćemo skup podataka svih trgovina u gradu Zagrebu koji se sastoji od ukupno 880 trgovina dok ćemo kao pandan skupu podataka koprivničko-križevačke županije koristiti trgovine u osječko-baranjskoj županiji. Skup podataka u osječko-baranjskoj županiji sastoji se od 170 trgovina, a tamo se nalaze i gradovi koji su po broju stanovnika, a time i trgovina, veći od gradova u koprivničko-križevačkoj županiji.

Skup podataka u gradu Zagrebu prikazan je na slici 19 dok je skup podataka u osječko-baranjskoj županiji prikazan na slici 20.



Slika 19 - skup podataka u gradu Zagrebu

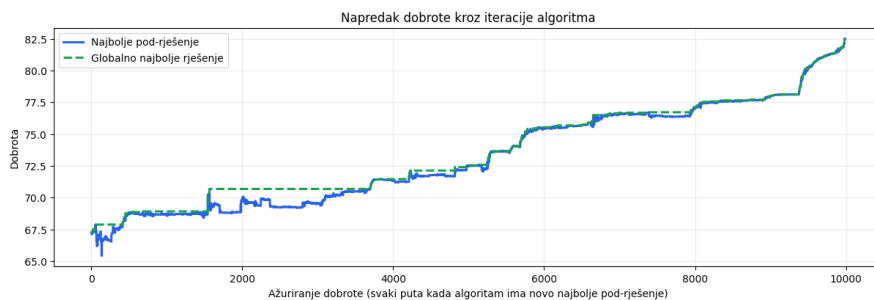


Slika 20 - skup podataka u osječko-baranjskoj županiji

6.2. Rezultati za grad Zagreb

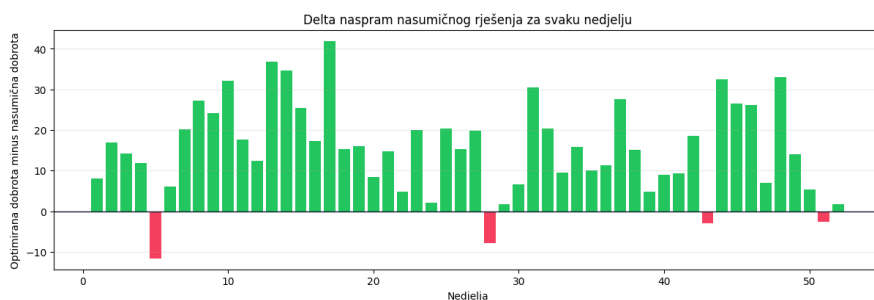
Ukupno trajanje optimizacije za grad Zagreb s navedenim hiperparametrima i uvjetima zaustavljanja pod-algoritma od 300 sekundi ili 2000 generacija ili stagnacijskim faktorom od 0.2 (400 generacija) iznosilo je približno 5400 sekundi, odnosno 90 minuta na računalu sa 4 procesorske jezgre. Za to vrijeme, algoritam je pronašao rješenje sa iznosom funkcije dobrote od 82.51 što je za 16.4 odnosno 25% bolje od prosjeka nasumičnih rješenja koji iznosi 66.1 u 1000 mjerenja. Napredak od 25% vrlo je značajan zato što se prevodi u srednje poboljšanje pokrivenosti od 16.4 km^2 svake nedjelje što je velika površina, pogotovo za grad kao što je Zagreb. Pregledom dijagrama konvergencije na slici 21 vidimo kako su rješenja pod-problema dobro pratila poboljšanje rješenja konačnog problema. Naime, dijagram konvergencije napravljen je na način da postoji promatrač koji promatra sve optimizacije koje se izvršavaju paralelno i dohvaća njihova najbolja rješenja. Kada od pod-optimizatora primi novo najbolje rješenje, on to rješenje ukomponira u svoj prototip konačnog rješenja i vrijednost funkcije dobrote takvog ukupnog rješenja sprema u arhivnu datoteku koju kasnije možemo analizirati. Razlog za ovakvu analizu jest činjenica da vrijednosti funkcije dobrote između paralelnih optimizacija nisu usporedive pa ih je potrebno promatrati u kontekstu konačnog, globalnog rješenja. Mana ovog pristupa je da se ne vidi vrijednost funkcije globalnog rješenja kroz sve iteracije algoritma. Naime, računanje

navedenoga bilo bi vrlo vremenski zahtjevno i došli bi do prepreke sinkronizacije kod paralelnog izvršavanja algoritma.



Slika 21 - dijagram napretka dobrote kroz iteracije algoritma za grad Zagreb

Analizom promjene funkcije dobrote na pojedinim nedjeljama na slici 22 s obzirom na neko nasumično rješenje vidimo kako je napredak u dobroti vidljiv u gotovo svim nedjeljama dok kod samo nekih dolazi do lošije zadovoljenosti prvog odnosno drugog kriterija.



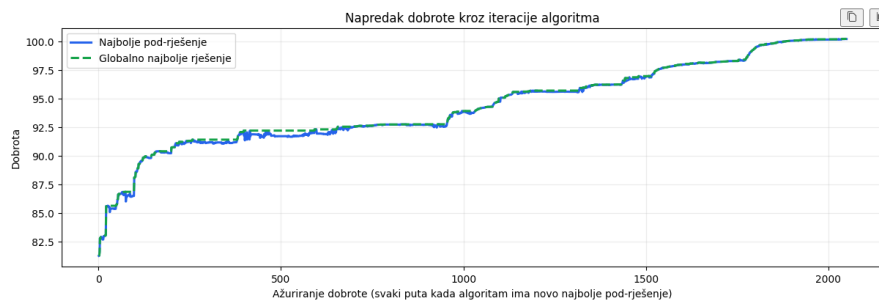
Slika 22 - dijagram promjene dobrote naspram nekog nasumičnog rješenja za svaku nedjelju

6.3. Rezultati za osječko-baranjsku županiju

Ukupno trajanje optimizacije za osječko-baranjsku županiju trajalo je 1540 sekundi odnosno gotovo 26 minuta na računalu s dvije procesorske jezgre što je puno brže od optimizacije za grad Zagreb. Razlog navedenom je izoliranost nekih trgovina što rezultira u manjim grupama koje konvergiraju brže kao i općenito manji broj trgovina u skupu podataka tako da je manja vremenska zahtjevnost očekivana.

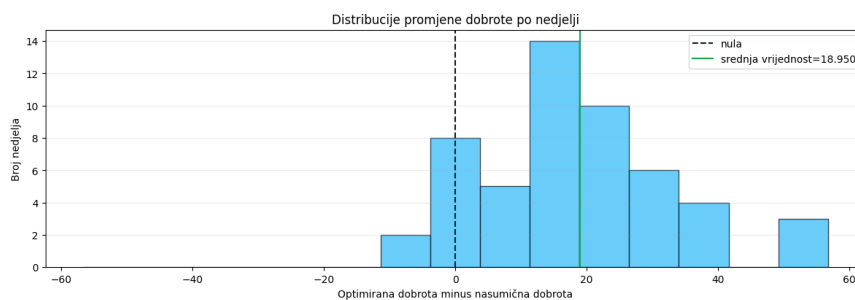
Za to vrijeme, algoritam je pronašao rješenje sa iznosom funkcije dobrote od 100.87 što je za 18.87 odnosno 21% bolje od prosjeka nasumičnih rješenja koji iznosi 82 u 1000 mjerenja. Pregledom dijagrama konvergencije na slici 23 primjećujemo kako najbolje globalno

najbolje pod-rješenje blisko prati najbolje pod-rješenje pod-algoritma. Razlog tome je što u ovom skupu podataka manji broj trgovina zapravo interagira pa su dobrote pod-problema često baš one konačne dobrote konačnog rješenja.



Slika 23 - dijagram napretka dobrote kroz iteracije algoritma za osječko-baranjsku županiju

Analiza promjene dobrote za pojedine nedjelje ponovno prikazuje kako se u velikom udjelu nedjelja dobrota pojedine nedjelje povećava dok se tek kod nekoliko njih ona smanji. Na dijagramu distribucije za promjenu dobrote po nedjelji na slici 24 vidimo kako je ovo slučaj. Ovo ponovno ukazuje na dobro ujednačeni rad algoritma.



Slika 24 - distribucija promjene dobrote po nedjelji za osječko-baranjsku županiju

6.4. Sličnost rješenja

Usporedili smo i sličnosti između nekoliko pokretanja prethodno navedenih skupova podataka koristeći mjeru Jaccardove sličnosti nad parovima $p = (trgovina, nedjelja)$ za trgovine koje rade neke nedjelje. Definirajmo neko rješenje R_i kao skup svih parova p koji opisuju rješenje našeg problema. Tada se Jaccardova sličnost $J(R_i, R_j)$ između rješenja i i j računa kao

$$J(R_i, R_j) = \frac{|R_i \cap R_j|}{|R_i \cup R_j|}$$

Dodatno, iz računa smo uklonili parove trgovina koji su osigurani od strane ograničenja kako bi sličnost računali samo uključujući parove koje je algoritam mogao dodavati i uklanjati.

U tablici 2 prikazane su sličnosti između nekih izmjerenih instanci optimalnih rješenja za navedene skupove podataka (ZG predstavlja skup trgovina u gradu Zagrebu, dok OB predstavlja skup trgovina u osječko-baranjskoj županiji). Iz podataka se može zaključiti kako više pokretanja identičnih instanci algoritma na skupu OB rezultira sličnoj dobroti rješenja. Međutim, Jaccardove sličnosti između rješenja vrlo su niske iz čega ne možemo definitivno zaključiti da algoritam konvergira u ista rješenja kod različitih i nasumičnih pokretanja.

Tablica 2 - usporedbe sličnosti rješenja

Rješenje 1	Rješenje 2	Dobrota rješenja 1	Dobrota rješenja 2	Sličnost rješenja 1 i 2
OB1	OB2	100.221	99.001	0.421
OB1	OB3	100.221	100.865	0.436
OB1	OB4	100.221	97.314	0.426
OB1	OB5	100.221	99.484	0.411
OB2	OB3	99.001	100.865	0.415
OB2	OB4	99.001	97.314	0.386
OB2	OB5	99.001	99.484	0.389
OB3	OB4	100.865	97.314	0.427
OB3	OB5	100.865	99.484	0.418
OB4	OB5	97.314	99.484	0.383
ZG1	ZG2	82.508	80.865	0.33

7. Zaključak

Commented [SD19]: UPDATE: Zaključak dodan. Ja sam jako zadovoljan zaključkom i ukupnim rezultatom 😊

Problem optimizacije rasporeda radnih nedjelja na prvi je pogled lakši nego u stvarnosti. Naime, s obzirom na velike količine znanstvenih radova koji se bave optimizacijom neke vrste rasporeda za očekivati bi bilo da je ovaj problem još samo jedan od njih. Međutim, ispostavlja se kako spajanje više znanstvenih domena, kao što su ovom diplomskom radu računarstvo i ekonomija, dovodi do veće složenosti problema.

Prva prepreka bio je dizajn matematičkog modela kojeg smo koristili za mjerenje kvalitete naših rasporeda. Naime, jasno je kako ne postoji točan i objektivni model koji će savršeno opisivati mikro-ekonomsko stanje nekog prostora tako da smo mi, kao i drugi znanstvenici kao što je Reilly, prisegli heurističkim pristupima koji su inspirirani stvarnim pojavama u prirodi i društvu kako bi barem nekako mogli modelirati naš problem. Mana heurističkog pristupa je to što u konačnici mi ni na koji način za naše rasporede ne možemo tvrditi da su optimalni, pa čak niti da su ista kvalitetniji od onih koje trgovci trenutno koriste.

Nakon dizajna matematičkog modela implementirali smo efikasan i generički model genetskog algoritma koji smo mogli koristiti za jednostavno eksperimentiranje i koji omogućuje kasniju izmjenu ishodišnog matematičkog modela. Za dodatno olakšanje korištenja algoritma implementirano je i korisničko i programsko sučelje. Generičnost implementacije genetskog algoritma iznimno se isplatila zato što smo se, nakon zahtjevne implementacije programskog okvira, u potpunosti mogli posvetiti dizajnu i implementaciji specifičnih genetskih operatora koji su prikladni za rješavanje problema optimizacije rasporeda.

Kao rezultat, dobili smo kompletan sustav koji je gotovo spreman za komercijalnu upotrebu, a koji na vrlo jednostavan način omogućuje trgovcu da s obzirom na svoje podatke, pronađe optimalan raspored u okviru našeg modela s dva kriterija. U okviru modela, sustav optimizacijom pronalazi značajno bolja rješenja od onih nasumičnih iz čega se može naslutiti kvaliteta implementiranog genetskog algoritma. Ipak, prava potvrda za kvalitetu našeg rada moguće je jedino dobiti implementacijom našeg rasporeda kod nekog trgovačkog lanca i mjerenja konkretnih metrika kao što su promet i prihod, a koji bi bili konkretni pokazatelji na poboljšanje poslovanja prikladnog trgovca u odnosu na prijašnje rasporede.

Literatura

Commented [SD20]: UPDATE: Literatura

- [1] Filipović, M. *Šef udruge malih trgovaca: "Kako biram 16 radnih nedjelja? K'o kokoš kad nabada zrno. Nekad pogodim, nekad ne"*, N1Info, (2024, studeni). Poveznica: <https://n1info.hr/biznis/sef-udruge-trgovaca-o-tome-kako-bira-radne-nedjelje>; pristupljeno 5. svibnja 2026.
- [2] Hrvatski Sabor, Zakon o izmjenama i dopuni Zakona o trgovini, Narodne Novine, (2023, ožujak), Poveznica: https://narodnenovine.nn.hr/clanci/sluzbeni/full/2023_03_33_580.html, pristupljeno 5. svibnja 2026.
- [3] Čupić, M. *Raspoređivanje nastavnih aktivnosti evolucijskim računanjem*, Doktorski rad, Sveučilište u Zagrebu Fakultet elektrotehnike i računarstva, 2011.
- [4] Perak, J. *Optimizacija rasporeda ispita na fakultetu primjenom metaheuristika*, Diplomski rad, Sveučilište u Zagrebu Fakultet elektrotehnike i računarstva, 2025.
- [5] Zhou, B. *Reilly's law of retail gravitation*, Southern Illinois University Edwardsville, (nepoznat datum), Poveznica: https://www.siu.edu/~bzhou/class/business_geography/markets/Reilly.html, pristupljeno 5. svibnja 2026.
- [6] Čupić, M. *Prirodom inspirirani optimizacijski algoritmi*, Sveučilište u Zagrebu Fakultet elektrotehnike i računarstva, (2009), Poveznica: <https://www.fer.unizg.hr/download/repository/Cupic2009-PrirodomInspiriraniOptimizacijskiAlgoritmi.pdf>, pristupljeno 5. svibnja 2026.

Izjava o korištenju umjetne inteligencije

Commented [SD21]: UPDATE: izjava o korištenju AI

U svrhu izrade diplomskog rada korišten je GPT-5.5 u obliku agentske implementacije dijelova korisničkog sučelja, inicijalne opise prostora pretraživanja, skripte za optimizaciju hiperparametara, implementaciju O'Reillyevog modela, implementaciju brze funkcije dobrote, implementaciju jupyter bilježnica za uredan prikaz podataka i generiranje skripti u okviru prikupljanja podataka s interneta. Za istraživanje, iteraciju dizajna implementacije, ideje za popravke grešaka i moguća proširenja sustava korišten je Gemini 3.1 Pro Preview model.

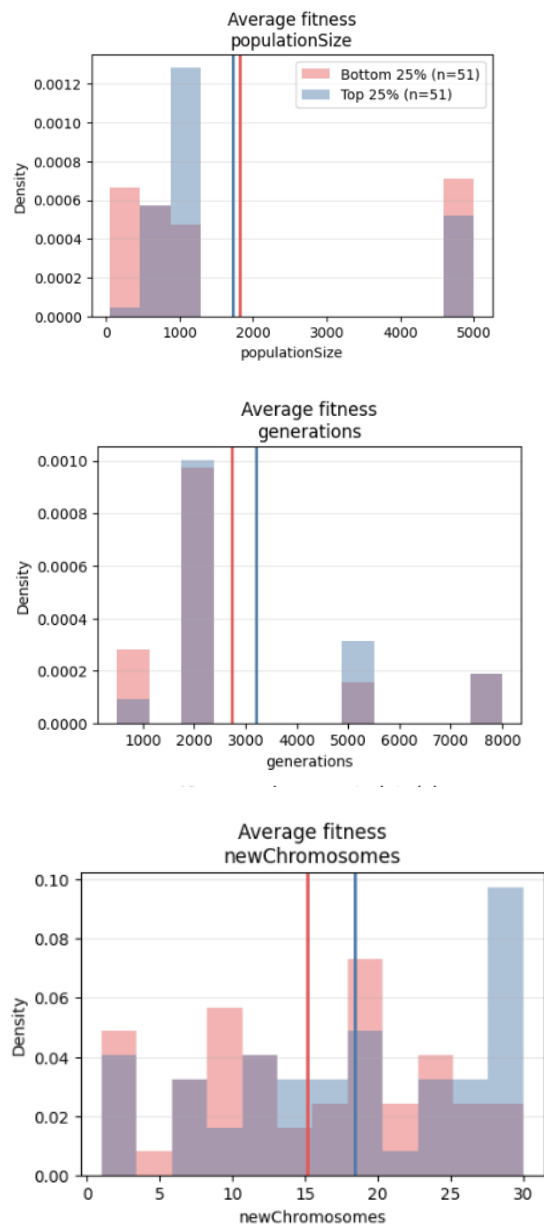
Osvrt na korištenje umjetne inteligencije: Modele umjetne inteligencije trebalo je navoditi u dobrom smjeru što se tiče infrastrukture implementacije kako bi programer ostao u toku s promjenama u bazi koda. Navedeno je dodatno olakšalo i ubrzalo proces implementacije zato što je agent umjetne inteligencije bio zadužen samo za krajnju implementaciju, ali ne i za dizajn sustava. Prethodno nije istina i za grafički dio korisničkog sučelja s obzirom da rad s grafičkim sučeljem nije dio svrhe izrade rada. Umjetna inteligencija često je previdjela greške u dizajnu ili napredne greške u logici pa je ljudska intervencija bila potrebna u navedenim slučajevima.

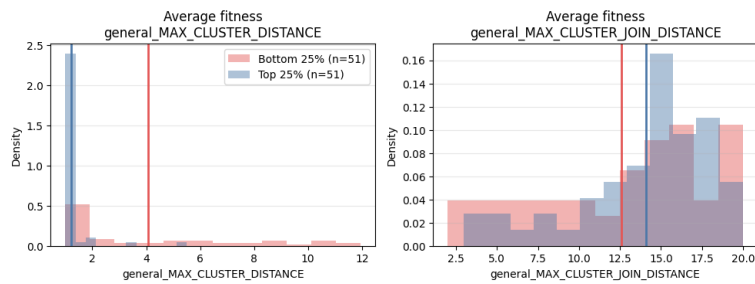
Izjava o korištenju umjetne inteligencije: Potvrđujem da sam tijekom izrade ovog završnog rada koristio umjetnu inteligenciju u skladu s Politikom primjerenog korištenja umjetne inteligencije na Fakultetu elektrotehnike i računarstva, te da umjetna inteligencija nije zamijenila moje kritičko razmišljanje niti moj doprinos.

Privitak

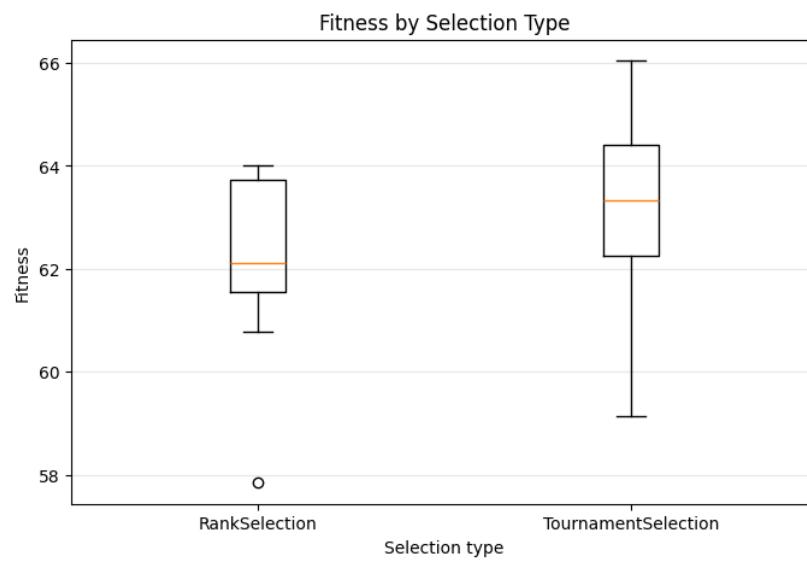
Commented [SB22]: UPDATE: attachmenti za analizu hiperparametara, previse plotova za poglavlje o optimizaciji hiperparametara.

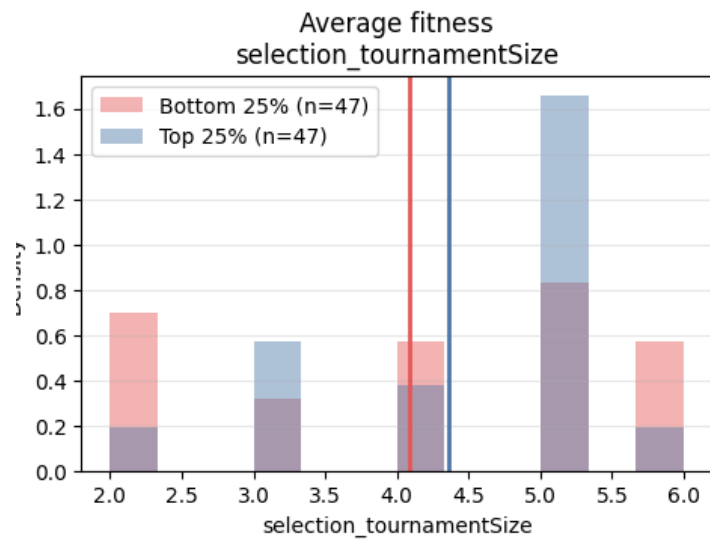
1. Optimizacija hiperparametara – općenite postavke



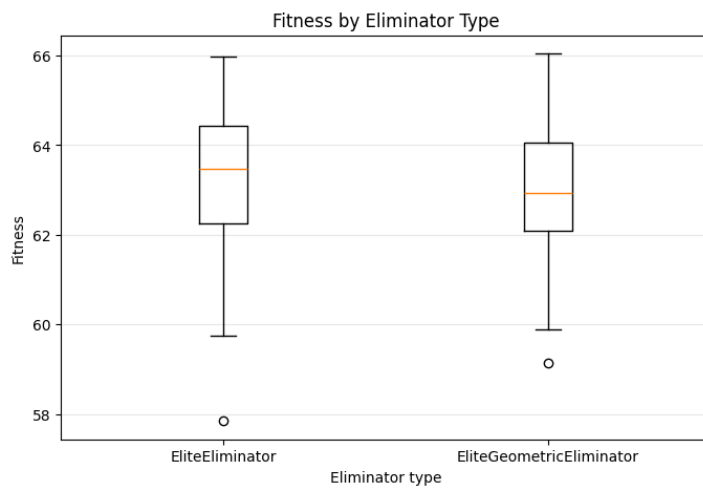


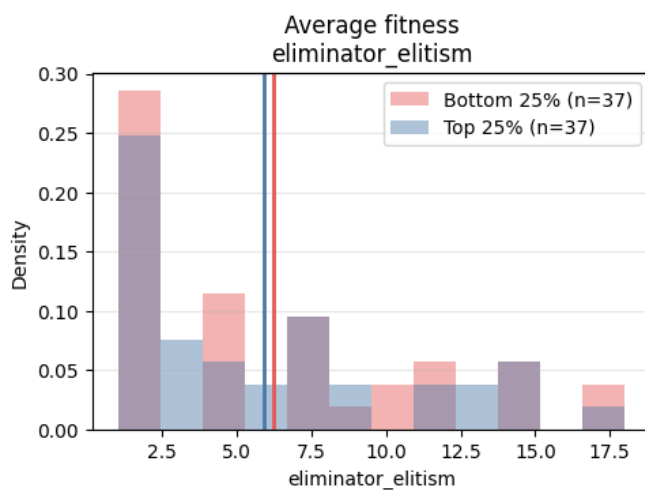
2. Optimizacija hiperparametara – selekcija





3. Optimizacija hiperparametara – eliminacija





4. Optimizacija hiperparametara – križanje

